

TDK-dolgozat

Szente Péter

Kinematikai útvonaltervezés akadályokkal teli parkolási helyzetekben

EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

NUMERIKUS ANALÍZIS TANSZÉK



Szerző:

Szente Péter

programtervező informatikus MSc

2. évfolyam

Témavezetők:

Dr. Lócsi Levente

egyetemi adjunktus

Eötvös Loránd Tudományegyetem

Budapest, 2025

Mócsai Tamás

Product Owner, Lead Engineer

Robert Bosch Kft.

Tartalomjegyzék

1. Bevezetés	3
1.1. Szakirodalmi áttekintés	4
1.2. Kutatási kérdések	6
1.3. A dolgozat tartalma, eredmények	6
2. Elemzés és tervezés	8
2.1. Mit nevezünk útvonaltervezésnek?	8
2.2. Útvonaltervezés formálisan	9
2.3. A jármű kinematikai modellje	11
2.4. Szabadtéri útvonaltervezés	12
2.5. Útvonaltervező algoritmusok	14
2.5.1. A* algoritmus	15
2.5.2. RRT	18
2.6. Sima útvonal létrehozása	19
2.7. Geometriai lekérdezések	20
3. Megvalósítás és alkalmazás	23
3.1. Heurisztikák	23
3.1.1. Egyszerű heurisztikák	23
3.1.2. Nemholonomikus heurisztikák	24
3.1.3. Objektumtudatos heurisztikák	25
3.2. Geometriai lekérdezések	26
3.2.1. Analitikus ütközésetektálás	27
4. Szimulációs eredmények	30
4.1. Szabványok	30
4.2. Tesztelés módja, metrikák	31
4.2.1. Hiperparaméter beállítás optimalizációval	31

4.3. Eredmények	32
5. Összefoglalás és eredmények	34
5.1. Nyilatkozat a használt eszközökről	35
Köszönetnyilvánítás	36
A. Függelék	37
A.1. Parkolási esetek	37
A.2. Reeds–Shepp-útvonalak	41
Irodalomjegyzék	43
Ábrajegyzék	48
Jelölésjegyzék	50

1. fejezet

Bevezetés

Az önvezető és vezetést segítő rendszerek az autóipar egyik vezető trendjévé váltak. Segítségükkel az autók biztonságosabbak lesznek, kevesebb energiát használnak, vagy kényelmi funkciókkal bővülnek. A korai vezetést segítő rendszerek fókusza a biztonság és a kényelem volt. Ilyenek például a blokkolásgátló rendszer, a sávtartás és a sebességtartás, illetve vészfékezés funkció, mely a frontális ütközést hivatott elkerülni az autó lelassításával. Az utóbbi években egyre több autót szerelnek fel nagy felbontású szenzorokkal (lidar, radar, kamera), melyek feltérképezik a jármű környezetét. Az önvezető funkciók feladata a rendelkezésre álló információk alapján a vezető inputjának hatására egy biztonságos manőver megtervezése és végrehajtása.

Ezen technológiák előfeltétele egy dinamikusan végrehajtható, biztonságos útvonal gyors megtervezése, mely figyelembe veszi a jármű kinematikai limitációit. Tetszőleges parkolási esetek megoldása továbbra is kihívást jelentő probléma, mely akadályokkal teli környezetben egy rendkívül számításigényes feladat. Habár a szakirodalomban több heurisztikus módszer ismert (A* algoritmus diszkrétizált állapottéren, RRT, spline-alapú módszerek, optimalizáción alapuló módszerek), több nyitott kérdés is van. Ilyen például a keresési tér diszkrétizációjának és a mozgási primitíveknek az optimális konfigurációja, olyan heurisztikák keresése, melyek egyszerre veszik figyelembe az akadályokat és az autó kinematikai limitációit, továbbá az útvonal újratervezése és a heurisztika frissítése abban az esetben, ha a szenzorfüzió új akadályt detektál.

A kutatás célja ezen problémák megoldására szolgáló korábbi algoritmusok elemzése és továbbfejlesztése, illetve új módszerek kidolgozása. A vizsgált alproblémák között szerepelnek: ütközésdetektáló algoritmusok, objektumokat is figyelembe vevő heurisztikák összehasonlítása és egy egyszerűsített kinematikai modellel való egyesítése, a mozgási

primitívek optimalizálásának lehetőségei, új heurisztikák és az útvonal gyors újratervezésének kérdése. Vizsgálandó szempont még a limitált számítási erőforrások hatékony kihasználása, mivel a módszerek tesztautóban is kipróbálásra kerülhetnek a jövőben.

1.1. Szakirodalmi áttekintés

Az útvonaltervezés az algoritmikus megoldások mellett már a kezdetektől fogva erősen támaszkodik fizikai és matematikai eredményekre. Mivel a cél egy fizikai korlátokkal rendelkező jármű navigálása, így fontos a *jármű modellezése*. Az egyik leggyakrabban alkalmazott modell az Ackermann modell [1]. Ez feltételezi, hogy a kerekek a differenciálhajtóműnek köszönhetően csúszásmentesen haladnak. Az egy tengelyen lévő kerekeket további egyszerűsítésként gyakran egynek veszik. Ekkor biciklimodellről [2] beszélünk. Ezen egyszerűsítések jól modellezik egy négykerekű jármű mozgását, ha a sebesség nem túl nagy. Polack és tsai. azonban rámutatnak [2], hogy bár az útvonaltervezésben ezzel a modellel követhető útvonalat kapunk, a kontrollerben már nem érdemes használni, mert az apró hibák a manőver végrehajtása során összeadódnak, és a jármű letér a tervezett útvonaltól.

Egy másik fontos modell az autó optimális hosszúságú útvonalát adja meg abban az esetben, ha nincsenek akadályok. Ezen *szabadtéri útvonaltervezés* megoldását Dubins írta le először [3] azzal a megkötéssel, hogy a jármű csak előre mehet. Fontos eredmény, hogy az optimális útvonal legfeljebb 3 útvonaldarabból áll, melyek egyenesek vagy adott sugarú körívek lehetnek. Ez lehetővé teszi, hogy a legrövidebb útvonalat egyszerű matematikai formulák segítségével kiszámítsuk. A teljes probléma megoldását Reeds és Shepp adták meg [4]. Az előzőhöz hasonló eredmény szerint az optimális útvonal legfeljebb 5 útvonaldarabból áll, melyek között legfeljebb 2 menetirányváltás is lehet.

Egy önvezető járműnél a környezet érzékelésére, a *percepcióra* széles szakirodalom elérhető. Yeong és tsai. egy viszonylag friss összefoglaló tanulmányban [5] összefoglalják a használt módszereket. A percepció rendszer alapvetően szenzorok kombinációjából áll, mint például LIDAR, radar, kamerák és ultrahangos érzékelők. A szenzorok segítségével a jármű képes felismerni a környezetében található akadályokat, a szabad parkolóhelyeket és az útfelület geometriáját. A szenzoros adatok összeolvasztásával (sensor fusion) pontos és megbízható környezetérzékelést próbálnak megvalósítani.¹ Az útvonaltervezés

¹ A percepcióval a dolgozat nem foglalkozik részletesen.

a szenzorfüzió komponens kimenete alapján tervezi az útvonalat, így fontos ezen lépés megbízhatósága.

Az akadályok elkerülésére különböző *geometriai* megközelítéseket alkalmaznak. Itt megkülönböztethetünk poligon alapú és mező alapú módszereket. A poligon alapú módszerek [6, 7–8. fejezet.] leginkább egy útvonal vagy pozíció ellenőrzésére alkalmasak. Általában bináris kimenetükből az derül ki, ütközik-e az útvonal.

A *mező alapú* módszerek optimalizációs feladatok megoldására alkalmasak. Az legismertebb mező alapú módszer a Voronoi-diagram alapú akadályelkerülés, amely a jármű akadályoktól való távolságát próbálja maximalizálni az útvonal minimalizálása mellett. Egy másik gyakran alkalmazott megoldás a potenciálmező-alapú megközelítés [7, 8], amely egy vektormezőt hoz létre a jármű mozgásának irányítására, elkerülve az akadályokat és vezetve a cél felé.

Az *útvonaltervezés* feladata ezen építőelemekből a kritériumoknak megfelelő útvonal elkészítése. Az algoritmusokat osztályozhatjuk keresés alapú és felfedezés alapú algoritmusokra [9]. A keresés alapú algoritmusok közül a legismertebb a hibrid A* [10] és variánsai. Ezek az algoritmusok a keresési térben egy heurisztika segítségével keresik meg az optimális útvonalat. A felfedezés alapú algoritmusok közül a legismertebbek az RRT-variánsok² [11]. Ezek az algoritmusok először véletlen lépésekkel felfedezik a keresési teret. Ha találnak megoldást, akkor utána azt több iteráción keresztül finomítják.

A tervezett útvonal görbülete gyakran tartalmaz szakadásokat [12], melyeket valamilyen *simítási eljárással* [9] javíthatunk. Gyakran használnak Bézier-görbéket [12] és spline interpolációt [13] sima útvonalak meghatározására. Mindkettőnél kihívás a görbület korlátozása [14, 15]. Ha a görbület egy intervallumon nagyobb, mint a maximális görbület vagy szakadási helye van, akkor a jármű letér a meghatározott pályáról. Vannak olyan görbék, melyek természetesen követik a kormányzás által meghatározott trajektoriát. A *klotoidok* [16] a görbület állandó sebességű változását feltételezik, így ideális illesztőgörbék az útvonaltervezés számára.

Még ha az útvonaltervező algoritmus a hosszban optimálishoz közeli és követhető utat ad, akkor is érdemes tovább javítani rajta [10]. Az útvonal gyakran tartalmaz felesleges kormányzásokat. Ezeket egy *lokális optimalizációs* eljárással kisimíthatjuk. Itt figyelembe kell venni a maximális megengedett görbületet és az objektumoktól vett távolságot is.

Legvégül az elkészült útvonalat az autónak végig kell követnie. Ezt a *kontroller* végzi a motor és a kormánymotor állapotának változtatásával a tervezett trajektória alapján. Itt

² Rapidly Exploring Random Trees

ismét fontos szerepet játszik a fizikai modell és a bizonytalansági tényezők figyelembe vétele. Liu és tsai. egy 2023-as tanulmányukban [17] részletesen bemutatják a probléma komplexitását. Habár a kontroll az információs láncban az útvonaltervezés után van, mégis erős kimeneti követelményeket támaszt a tervezéssel szemben.

1.2. Kutatási kérdések

A dolgozat motivációjául az alábbi kutatási kérdések szolgáltak:

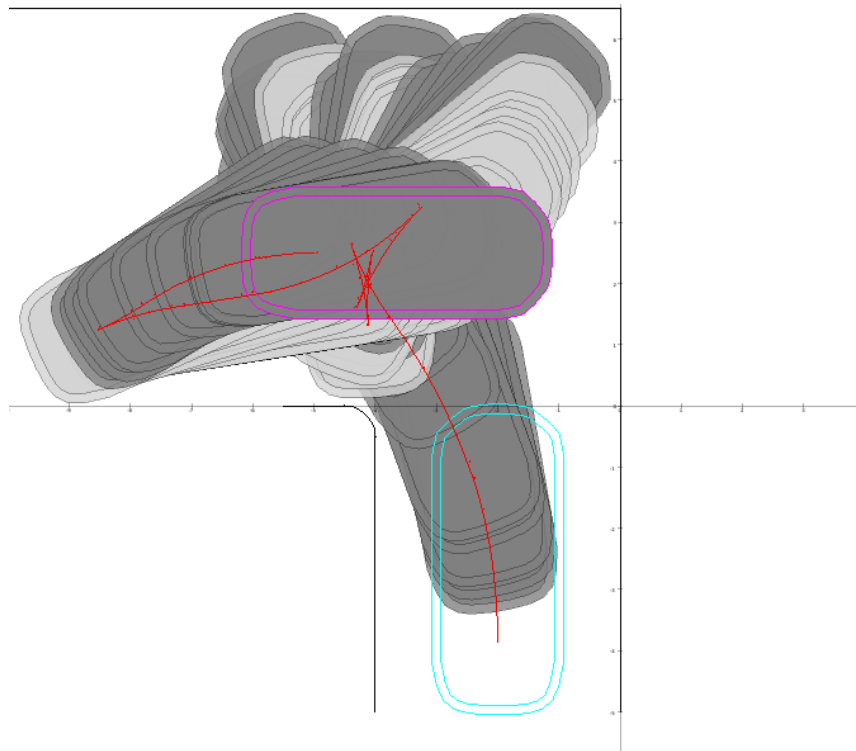
1. Melyek az önvezető autóknál használt legkorszerűbb útvonaltervező módszerek?
2. Hogyan lehet a keresési teret leszűkíteni az útvonal romlása nélkül?
3. Mi a mozgási primitívek optimális halmaza hatékonyság és útvonalminőség szempontjából?
4. Érdemes-e akadályokat is figyelembe vevő heurisztikát alkalmazni?
5. Melyek a költséges geometriai lekérdezések az útvonaltervezés esetében, és hogyan gyorsíthatók fel?

1.3. A dolgozat tartalma, eredmények

A dolgozat bemutatja az útvonaltervezéssel kapcsolatos legfontosabb fogalmakat és definíciókat. A 2. fejezetben széles körű áttekintést adok a nemholonomikus járművek útvonaltervezésénél használt legfontosabb módszerekről. Bemutatok egy, a tudomány jelen állása szerinti legjobb útvonaltervezőt, melyet önvezető autók parkolására optimalizálok. A 3. fejezetben bemutatom a rendszer továbbfejlesztésének lehetőségeit.

A főbb eredmények közé tartoznak:

1. A keresési tér és a primitívek optimális konfigurációja szimuláció segítségével.
2. Hatékony analitikus geometriai lekérdezések leírása és implementációja.
3. Módszerek a Reeds–Shepp-heurisztika hatékony kiszámítására.
4. Egy objektumtudatos heurisztika kiértékelése.



1.1. ábra. Egy kihívást jelentő parkolási esethez, a blokkolt merőleges parkoláshoz tervezett útvonal 6 irányváltással

2. fejezet

Elemzés és tervezés

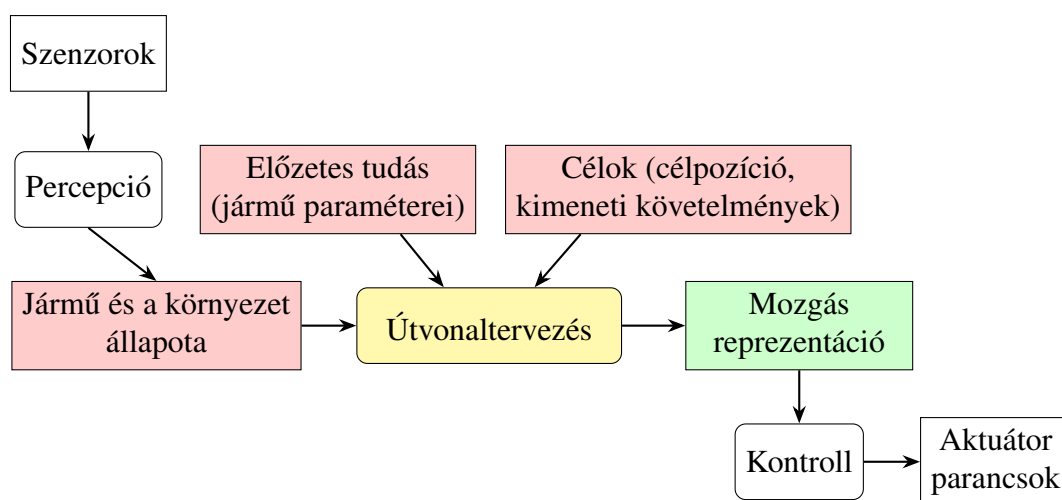
Ebben a fejezetben definiálom az útvonaltervezés legfontosabb fogalmait, illetve széles körű áttekintést adok a nemholonomikus járművek útvonaltervezésénél használt legfontosabb módszerekről.

2.1. Mit nevezünk útvonaltervezésnek?

1. Definíció. A *kinematikai útvonaltervezés* során a cél számítási módszerek alkalmazásával két térbeli konfiguráció közötti folytonos útvonal meghatározása az összes fizikai korlát betartása és az akadályok elkerülése mellett.

Ez egy tisztán geometriai probléma, amely mellőzi az időbeli és dinamikai komplexitást. Önvezető járműveknél ezt általában nem tehetjük meg, azonban ha feltesszük, hogy a jármű alacsony sebesség mellett hajtja végre a manővereket, akkor a feltételezéseink jól modellezik a valóságot [2]. Ezt az egyszerűsítést elfogadjuk, hiszen a dolgozat fő fókuszsa a parkolás. Ebben az esetben körültekintően kell navigálni a sok akadály és a rossz láthatóság, pontosabban az objektumok takarása miatt.

A probléma megoldásához fontos elkülönítenünk az útvonaltervező komponens hatáskörét. Ezt szemlélteti a 2.1. ábra. Az útvonaltervezésnek nem része a szenzoradatok feldolgozása és címkézése. Ezt a percepció komponens végzi, melynek része a szenzorfüziós hely- és helyzetmeghatározás is. A percepció kimenete a környezetben található *objektumok* járműhöz viszonyított relatív pozíciója és az objektumok fajtája. Ezek tipikusan lehetnek alacsony objektumok (pl. fekvőrendőr, útburkolati jelek), melyek felett áthajthatunk, és magas objektumok, melyeket mindenképp ki kell kerülnünk. A későbbiekben csak a magas objektumokkal foglalkozunk és *akadályoknak* hívjuk őket.

2.1. ábra. Önvezetés komponensdiagrammja¹

Az útvonaltervező ismeri továbbá a jármű paramétereit. Ezek közül az egyik legfontosabb a jármű *kontúrja* (felülnézeti, kétdimenziós körvonala). A másik legfontosabb paraméter a legszűkebb kanyarodási manőver, amit végre tudunk hajtani. Ez utóbbi a kinematikai modellből származó számunkra legfontosabb paraméter.

Az útvonaltervezés egy költséges művelet, így mindig egy jól definiált céllal hajtjuk végre. Ez tipikusan egy útvonal megtalálása a jelenlegi vagy egy prediktált kezdőpozícióból egy célpozícióba. A célok között lehetnek megkötések is. Ezek gyakran a menetirányra vagy az útvonal simaságára vonatkoznak.²

Az útvonaltervező valamilyen útvonal-reprezentációt állít elő, mely összeköti a kiindulási- és a célpozíciót. Ezt aztán a kontroller hajtja végre. Ezen modul feladata a trajektória lekövetése a jármű aktuátorainak aktiválásával, és a kisebb hibák korrigálása. A kontroller nem fog tudni bármilyen útvonalat lekövetni, így fontos, hogy a tervezett útvonal *vezethető* legyen. Ezen kimeneti követelményt általában az útvonal bizonyos szintű simaságával és folytonosságával formalizáljuk.

2.2. Útvonaltervezés formálisan

A következőkben bevezetek néhány fogalmat és jelölést. Ezek a robotikából átvett szokványos definíciók.

¹Leegyszerűsített modell Kris Hauser vizualizációja alapján.

² Például, ha a jármű előre mozgásban van, akkor az útvonal nem kezdődhet hátramenettel. Egy másik gyakori feladatban arra vagyunk kíváncsiak, elérhető-e a célpozíció egy egyszerű manőverrel. Ilyenkor az útvonal egyáltalán nem tartalmazhat hátramenetet.

Legyen $\mathcal{C}$ a *Konfigurációs tér*. Ez tartalmazza a jármű állapotát leíró vektor összes lehetséges állapotát. Például, ha a pozíciót és az irányt szeretnénk reprezentálni akkor $\mathcal{C} = \mathbb{R}^2 \times [-\pi, \pi)$.

Jelöljük a *munkaterületet* $\mathcal{W}$ -vel. Ez tartalmazza a tér összes pontját, melyben az objektumok és a jármű (vagy robot) fizikailag elhelyezkedik. Például ha felülnézetből szeretnénk útvonalat tervezni, akkor $\mathcal{W} = \mathbb{R}^2$.

Legyen a jármű által lefedett munkaterület egy $\mathbf{q} \in \mathcal{C}$ konfigurációban $\mathcal{K}(\mathbf{q}) \subset \mathcal{W}$. Itt fontos megjegyeznünk, hogy nincs egyértelmű megfeleltetés a konfigurációs tér és a munkaterület között, hiszen a lefedett munkaterület meghatározásához ismernünk kell a jármű alakját. A későbbi egyszerűbb megfogalmazás miatt $\mathcal{K}(\cdot)$ határvonalát *kontúrnak* nevezzük.

Legyen $\mathcal{E}$ a környezet által lefedett munkaterület. Ez a halmaz olyan objektumokat reprezentál, melyeket mindenképp el kell kerülnünk az útvonaltervezés során. Ezen fogalmak segítségével definiálhatjuk a szabad tér fogalmát.

2. Definíció. A *szabad tér* a konfigurációs tér azon pontjait tartalmazza, melyekben a jármű által lefedett munkaterület nem metszi a környezetet. Formálisan:

$$\mathcal{F} = \{\mathbf{q} \in \mathcal{C} \mid \mathcal{K}(\mathbf{q}) \cap \mathcal{E} = \emptyset\}.$$

Legyen $p(t) : [0, 1] \rightarrow \mathcal{C}$ egy útvonal.

3. Definíció. Egy útvonal *ütközésmentes*, ha minden pontja a szabad térben van.

$$\forall t \in [0, 1] : p(t) \in \mathcal{F}$$

4. Definíció. Egy útvonal *végrehajtható* (járműveknél vezethető), ha ütközésmentes és minden kinematikai és dinamikai megkötésnek megfelel.

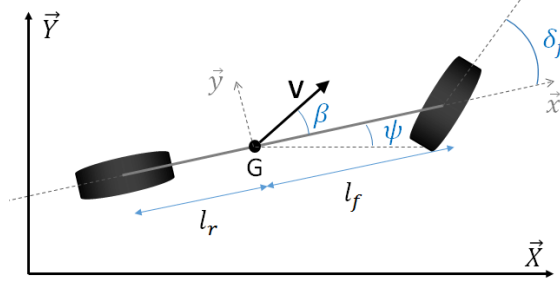
Az útvonaltervezés problémát tehát a

1. konfigurációs tér ($\mathcal{C}$),
2. munkaterület ($\mathcal{W}$),
3. kezdő- és célkonfigurációk ($\mathbf{q}_S, \mathbf{q}_T$),
4. a környezet ($\mathcal{E}$),
5. és a kinematikai korlátok (később)

együttesen határozzák meg.

2.3. A jármű kinematikai modellje

A járművek kinematikai modelljével ebben a dolgozatban részletesen nem foglalkozom, mivel a témának széles szakirodalma [2] van. A 2.2. ábrán az útvonaltervezésben leggyakrabban alapnak vett biciklimodell szerepel.



2.2. ábra. Kinematikai bicikli modell [2]

A modellből számunkra fontos megállapítás, hogy a kormány elfordulása, a kerék elfordulása és az autó kanyarodásának pillanatnyi görbülete egyértelmű megfeleltetésben állnak egymással. Az autó kanyarodását két mennyiséggel, a kanyarodás sugarával ($r(t)$) és a görbülettel ($\kappa(t)$) szoktuk jellemezni. $r(t)$ megadja, hogy a jármű mekkora sugarú köríven menne körbe, ha a kerékszög nem változna. A két mennyiség között inverz reláció áll fent:

$$r(t) = \frac{1}{\kappa(t)}.$$

Mivel a dinamikai komplexitással az útvonaltervezés csak a megszorítások szintjén foglalkozik, a kinematikai modell legfontosabb paraméterei a minimális kanyarodási sugar (r_{min}) és az ennek megfelelő maximális görbület ($\kappa_{max} = \frac{1}{r_{min}}$).

Fontos megjegyeznünk, hogy a teljes rendszert nem lehet leírni holonomikus megszorításokkal [18]. Pontosabban, nincs olyan $f(\mathbf{q}) = 0$, ($\mathbf{q} \in \mathcal{C}$) egyenlet, amivel leírhatnánk az összes kinematikai megkötést.

Ez alkalmazásban azt jelenti, hogy a teljes állapot leírásához azt is ismernünk kell, hogyan jutottunk el egy adott állapotba. Például ha egy (x, y, θ) állapotban vagyunk (ahol θ a jármű iránya), akkor a kerék ψ szöge a korábbi útvonal függvényében többféle lehet. Ha egyenesen haladtunk, akkor $\psi = 0$, ha viszont kanyarodtunk, akkor ettől eltérő. Ezen megkötés miatt az útvonaldarabok egymáshoz illesztésekor a modell összes korlátozását figyelembe kell venni, és nem elegendő a paraméterekre tett korlátok (pl. κ_{max}) betartása.

2.4. Szabadtéri útvonaltervezés

Szabadtéri útvonaltervezés problémának nevezzük, amikor a környezet üres ($\mathcal{C} = \emptyset$), azaz nincsenek akadályok, melyeket ki kéne kerülnünk. Ez önmagában is egy bonyolult optimalizációs feladat lenne, azonban több matematikus munkájának köszönhetően ismerünk analitikus megoldásokat néhány egyszerűbb járműmodellre. Ezen modellek nem foglalják magukba a jármű teljes dinamikai komplexitását, azonban jól közelítik azt alacsony sebességű manővereknél, mint a parkolás.

Az első eredmény Dubins nevéhez kötődik. A cikkében [3] szereplő ún. Dubins-autó csak előre tud menni, és a kanyarodása egy adott κ_{max} érték által korlátolt. A cikkben bizonyította, hogy ezen autó legrövidebb útvonala két pont között csak egyenes szakaszokból és maximális görbületű körívekből állhat, melyek száma legfeljebb 3. Pontosabban, az optimális útvonal csakis CSC vagy CCC alakú lehet, ahol C egy körívet, S pedig egy egyenes szakaszt jelöl. Ezt nevezzük *Dubins-útvonallnak*.

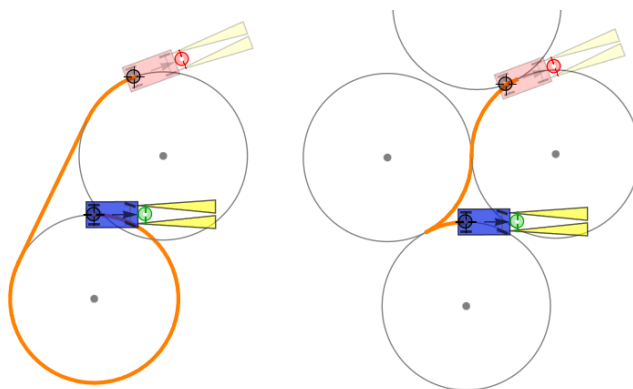
Ez az eredmény jelentősen lecsökkenti a kipróbálandó útvonalak számát. A szerző egyszerű zárt képletet is ad az útvonaldarabok hosszának meghatározására. A későbbiekben ezeket az útvonaldarabokat a szakirodalommal összhangban *mozgási primitíveknek* fogom hívni.

Egy másik hasonló, de sokkal jobban alkalmazható eredmény a *Reeds–Shepp-útvonal* [4]. A szerzők egy olyan járművet feltételeztek, mely előre és hátra is azonos költséggel tud közlekedni és a Dubins-autóhoz hasonlóan kanyarodási görbületben limitált. A cikk szerint egy ilyen autó legrövidebb útvonala két pont között az előbb említett C és S primitívekből, illetve irányváltásokból áll és legfeljebb 5 primitívet és 2 irányváltást tartalmazhat. A szerzők itt is megadják hogyan lehet analitikusan kiszámítani ezeket az útvonaltípusokat.

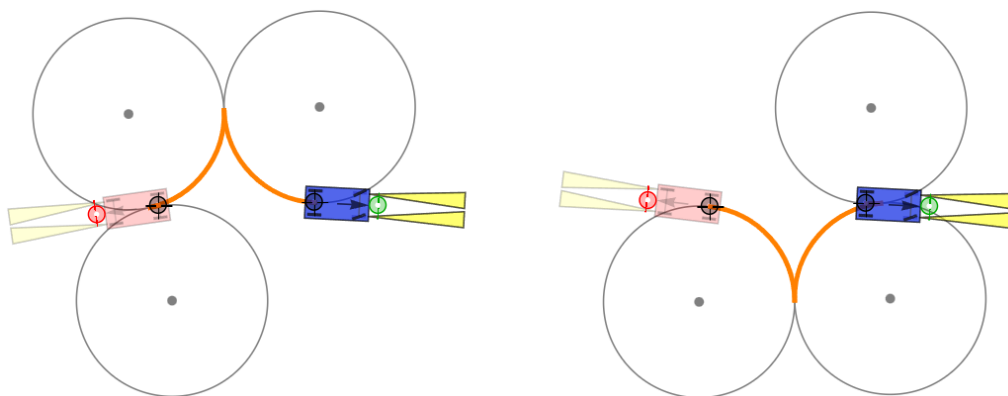
A Reeds–Shepp-útvonal gyakran megegyezik a Dubins-útvonallal, emellett viszont több esetben sokkal rövidebb. A Reeds–Shepp-útvonal további előnyös tulajdonsága, hogy az útvonal hossza a célkonfiguráció függvényében folytonos. Ennek következménye, hogy egymáshoz közeli célkonfigurációkhoz vezető útvonalak **hossza** közel lesz egymáshoz. A Dubins- és a Reeds–Shepp-útvonalak közti különbséget szemlélteti a 2.3. ábra.

Fontos megjegyezni, hogy az útvonalak nem pontonként folytonosak. Egymáshoz közeli célkonfigurációkhoz vezető legrövidebb útvonalak gyakran más mozgási primitíveket tartalmaznak. Ezt szemlélteti a 2.4. ábra.

Ezen útvonalak hosszban optimálisak, de nagy problémájuk, hogy egy egyszerű kont-



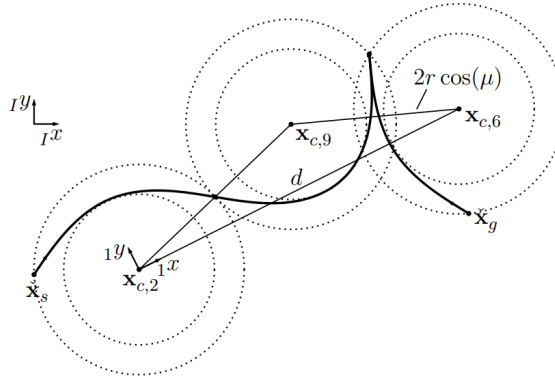
2.3. ábra. Dubins-útvonal (bal) és Reeds–Shepp-útvonal (jobb) összehasonlítása [19]



2.4. ábra. Egymáshoz közeli célkonfigurációkhoz vezető legrövidebb Reeds–Shepp útvonalak lehetnek nagyon különbözők [19]

roller nem tudja őket lekövetni. Ennek oka, hogy amikor egyenes szakasz után körív következik, az útvonal görbülete nem folytonos. Ez azt jelentené, hogy a kereknek azonnal egyenes állásból a maximális elfordulásba kellene váltani. Ez technikailag ugyan lehetséges, de csak álló autó esetében.

Az útvonalat görbületben kisimíthatjuk, ha két különböző görbületű primitív közé klotoidokat építünk be. Ennek egyik változata a folytonos görbületű *CC steer* [20]. Egy másik változat, a hibrid görbületű *HC steer* [21] megengedi a görbület hirtelen változását, de csak irányváltáskor, amikor egyébként is meg kell állni. Ez sokat segít olyan parkolási esetekben, ahol sokszor kell irányt váltani egy szűk helyen. A *CC steer* esetet szemlélteti a 2.5. ábra. Az ábrán jól látszik a módszer hiányossága is, ugyanis irányváltásnál lényegesen nagyobb helyre van szükség egy *CC* manőver végrehajtásakor, mint a Reeds–Shepp esetben.



2.5. ábra. CC steer primitívekből álló útvonal (2.15. ábra Banzhaf disszertációjából [22])

2.5. Útvonaltervező algoritmusok

Az útvonaltervező algoritmus feladata a bemeneti és kimeneti követelmények (2.2) figyelembe vételével a környezet felfedezése és egy megfelelő útvonal megtalálása a kiinduló- és a célállapot között. Az általános célú útvonaltervező algoritmusokat két nagyobb csoportra bonthatjuk [9]. Az egyik a gráfkeresés alapú tervezők, mint a Hibrid A* és változatai. A másik nagy csoport a mintavételezés alapú tervezők, mint az RRT* és variánsai. A továbbiakban definiálok néhány fogalmat, majd segítségével bemutatom a tervezők legfontosabb tulajdonságait.

5. Definíció. Egy útvonaltervező algoritmus *teljes*, ha véges időn belül terminál és ad egy útvonalat, amennyiben az létezik. Ha nem létezik útvonal, akkor azt jelzi.

6. Definíció. Egy útvonaltervező algoritmus *optimális*, ha véges időn belül terminál és az optimális útvonalat adja, amennyiben az létezik. Ha nem létezik útvonal, akkor azt jelzi.

1. Tétel. Az *optimális útvonaltervezők teljesek*.

Bizonyítás. A definíciókból következik. □

Ezek a definíciók természetesek, de sajnos néhány egyszerű esettől eltekintve teljes és optimális útvonaltervező algoritmus nem ismert. Éppen ezért bevezetjük az alábbi, megengedőbb fogalmakat, melyek a „majdnem optimális” fogalmát formalizálják.

7. Definíció. Egy útvonaltervező algoritmus *felbontás-teljes*, ha véges időn belül terminál és egy $h > 0$ felbontás paraméter mellett ad egy útvonalat, amennyiben az létezik. Ha nem létezik útvonal $h > 0$ felbontás paraméterrel, akkor azt jelzi. Ha létezik útvonal, a felbontás-teljes útvonaltervező talál útvonalat amennyiben a h paraméter megfelelően kicsi.

Ezt a definíciót akkor alkalmazzuk, amikor valamilyen paramétert diszkrétizálunk. Ez lehet a munkatér diszkrétizációja ütközésetektáláshoz vagy a paraméterter diszkrétizálása az implicit állapotgráf méretének csökkentéséhez. Ha a tervező nem talál útvonalat, akkor nem lehetünk benne biztosak, hogy nem létezik, vagy h -t túl nagyra választottuk.

8. Definíció. Egy útvonaltervező algoritmus *probabilisztikusan teljes*, ha annak a valószínűsége, hogy nem talál útvonalat, a tervezés előrehaladtával 0-hoz konvergál.

Ezt a definíciót a mintavételezés alapú tervezőknél használjuk.

9. Definíció. Egy útvonaltervező algoritmus *közéltően optimális*, ha a talált útvonal legfeljebb az optimális útvonal $(1 + \varepsilon)$ -szerese valamilyen $(\varepsilon > 0)$ konstansra.

10. Definíció. Egy útvonaltervező algoritmus *aszimptotikusan optimális*, ha annak a valószínűsége, hogy a talált útvonal hossza legfeljebb az optimális útvonal $(1 + \varepsilon)$ -szerese valamilyen $(\varepsilon > 0)$ konstansra, 1-hez konvergál.

Azaz az útvonaltervező tetszőlegesen jó útvonalat talál a tervezés előrehaladtával.

2.5.1. A* algoritmus

Az A* algoritmus egy gráfkeresésen alapuló útvonaltervező algoritmus. Az útvonaltervezés során gyakran implicit gráfokat készítünk a paraméterterben, és ezen futtatjuk a tervezőt. Rengeteg variánsa és kiegészítése létezik, melyet Paliwal jól összefoglal összehasonlító tanulmányában [23]. Az alap A* algoritmus az 1. algoritmusban látható.

Az algoritmus egy Open és egy Closed listát tart nyilván az elért és már lezárt csúcsokból. Minden iterációban az Open listából kiveszi a legjobb elemet, majd kiterjeszti, azaz meghatározza a szomszédait és az Open listához adja őket. Az algoritmusban a $g(n)$ függvény az eddig ismert legrövidebb utat reprezentálja a start csúcsból az n csúcsba. Az algoritmushoz használt heurisztikát a $h(a, b)$ függvény jelöli egy a és b csúcs között.

Az algoritmus felfedez minden n csúcsot melyre

$$f(n) = g(n) + h(n, \text{target}) \leq g^*(\text{target}),$$

ahol $g^*(n)$ az optimális útvonal hossza start-ból n -be. A futási időre így a következő becslést adhatjuk: $T_{A^*} \leq O(b^d)$, ahol b az ún. branching factor, azaz a gráf átlagos kifoka csúcsként, és d a megoldás mélysége a gráfban. Az algoritmusban fontos szerepet játszik a heurisztika ugyanis ezzel érjük el, hogy a keresés nagy része a célcsúcs felé haladjon. A heurisztikára azonban vannak megkötések.

1. Algoritmus A* algoritmus**Funct** AStar(start, target, $\mathcal{E}$)

```

1: OpenList  $\leftarrow$  [start]
2: ClosedList  $\leftarrow$  []
3:  $g(\text{start}) \leftarrow 0$ 
4:  $f(\text{start}) \leftarrow g(\text{start}) + h(\text{start}, \text{target})$ 
5: while OpenList  $\neq \emptyset$  do
6:    $n \leftarrow \min_f(\text{OpenList})$  ▷ Legjobb csúcs a felfedezettek közül
7:   if  $n = \text{target}$  then return ClosedList
8:   end if
9:   OpenList  $\leftarrow$  OpenList  $\setminus \{n\}$ 
10:  ClosedList  $\leftarrow$  ClosedList  $\cup \{n\}$ 
11:  for all neighbors  $n'$  of  $n$  not in  $\mathcal{E}$  do ▷ Ütközésetektálás
12:     $\text{cost}_{n'} \leftarrow g(n) + \text{distance}(n, n')$ 
13:    if  $n' \in \text{ClosedList}$  then
14:      continue
15:    else if  $n' \in \text{OpenList}$  and  $\text{cost}_{n'} < g(n')$  then
16:      OpenList  $\leftarrow$  OpenList  $\setminus \{n'\}$ 
17:    else if  $n' \in \text{ClosedList}$  and  $\text{cost}_{n'} < g(n')$  then
18:      ClosedList  $\leftarrow$  ClosedList  $\setminus \{n'\}$ 
19:    else
20:      OpenList  $\leftarrow$  OpenList  $\cup \{n'\}$ 
21:    end if
22:     $g(n') \leftarrow \text{cost}_{n'}$ 
23:     $f(n') \leftarrow g(n') + h(n', \text{target})$  ▷ Hurisztika számítás
24:  end for
25: end while

```

11. Definíció. Egy heurisztika *admisszibilis*, ha minden csúcsra a hátralévő távolság alsó becslését adja. $\forall n : h(n, \text{target}) \leq g^*(n, \text{target})$ és $g^*(a, b)$ az optimális útvonal hossza a -ból b -be.

12. Definíció. Egy heurisztika *konzisztens* vagy *monoton*, ha $h(\text{target}) = 0$ és $h(a) \leq g^*(a, b) + h(b)$.

Ha a heurisztika *admisszibilis* és *konzisztens*, akkor a minimális $f(n)$ értékű csúcs-hoz vezető út minden lépésben optimális lesz. Amikor elérjük a célállapotot, akkor egy optimális utat kapunk.

Az algoritmusnak sok variánsa létezik, melyek mind az alap (1.) algoritmushoz tesznek hozzá valamit. Ezen variánsok ötletei általában szabadon kombinálhatók egymással. Alább egy listát közlök azon variánsokról, melyeket felhasználtam:

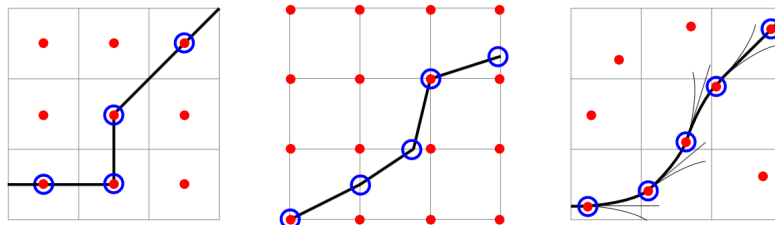
1. Az algoritmus minden csúcsot a memóriában tárol. Vannak *memóriakorlátos* variánsok, melyek a nagy memóriaigényt lecsökkentik az optimalitás elvesztése nélkül:

Iterative deepening A* [24], Simplified Memory Bounded A* [25].

2. Más variánsok konstans szorzóval növelik a heurisztika értékét [26], ez a *relaxáció*. Ilyenkor az útvonal hossza az optimálishoz képest a relaxáció mértékével arányosan növekedhet. Akkor érdemes használni, ha gyorsan kell útvonalat találni, de az optimalitás kevésbé fontos. Gyakorlatban jelentősen csökkenti a tervezés költségét, és gyakran az optimálishoz közeli útvonalat ad.
3. Azt is kihasználhatjuk, hogy a kiindulási- és célpozíciót is ismerjük. Ha mindkét irányból elindítjuk a keresést, akkor általában kevesebb kiterjesztéssel találunk útvonalat. Ezen *kétirányú* módszer ismert problémája volt, hogy nem feltétlenül ad optimális útvonalat. A heurisztikára tett további megkötésekkel azonban helyreállítható az optimalitás [27].
4. Az útvonaltervezés közben vagy az útvonal végrehajtásakor előfordulhat, hogy a szenzorrendszer új akadályokat detektál. Ilyenkor *dinamikus*an újra szeretnénk tervezni az útvonalat úgy, hogy a korábbi számítások minél kisebb részét kelljen invalidálni. Ezt oldja meg a D* lite [28] és variánsai.

Hibrid A*

Járművek útvonaltervezésénél általában implicit gráfon tervezünk útvonalat, melyeknél az egy pontból mozgási primitívvel elérhető állapotok száma végtelen. A *hibrid A** algoritmus [10] egy gyakran használt módszer diszkretizált állapot térben történő útvonaltervezésre. Az ötlet, hogy a nemholonomikus megkötéseknek eleget tevő primitívek közül olyanokat választunk, melyek legalább egy előre meghatározott h távolságra vannak egymástól. A ClosedList-ben tehát lezártnak tekintünk minden olyan csúcsot, mely egy már lezárt csúcs h sugarú környezetében van. Ezt az ötletet szemlélteti a 2.6. ábra.



2.6. ábra. Hibrid A* algoritmus állapottrácsa a jobb oldalon ([10, 2. ábra.])

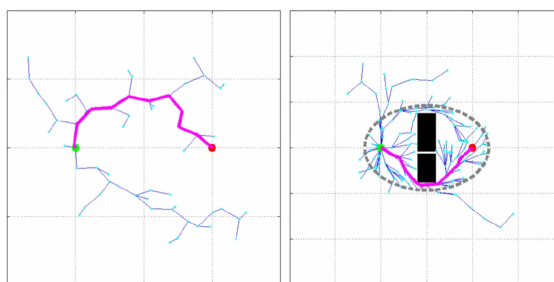
Az algoritmus admisszibilis és konzisztens heurisztika esetén felbontás-teljes és közelítően optimális. Ennél jobb elméleti tulajdonságokkal rendelkező útvonaltervező algoritmusokat csak speciális esetekben ismerünk.

Az algoritmus a cellák között valamilyen mozgási primitíven mozog, melynek vezethetőnek kell lennie. Gyakran használt primitívek az egyenesek és körívek gyors számíthatóságuk és egyszerű kezelésük miatt. Használatosak még a klotoidok is, ha görbületben sima útvonalat szeretnénk létrehozni.

A Hibrid A* és a korábbi variánsok kombinációjával elkészíthető egy állapottrácson alapuló, memóriakorlátos, relaxált, kétirányú, dinamikus hibrid A* tervező. Ezen algoritmus hatékony implementációja, optimális paraméterezése, tesztelése és validálása kihívást jelentő feladat, melyet a következő (3.) fejezetben részletezek.

2.5.2. RRT

A teljesség érdekében érdemes megemlíteni az RRT-hez hasonló, mintavételezés alapú tervezőket [11]. Ezek mintavételezik a konfigurációs teret és egy véletlen irányba terjesztik ki a fát az eddig elért útvonalakból. Esetenként rövidítik az útvonalat valamilyen heurisztika szerint. A tervezők probabilisztikusan teljesek és aszimptotikusan optimálisak, viszont általában nagyon lassan konvergálnak az útvonal megtalálása után, így leállításkor szuboptimális, és sok esetben nem végrehajtható útvonalat kapunk (2.7. ábra).



2.7. ábra. Informált RRT* algoritmus [29] által talált útvonalak finomítás előtt.

Nemholonomikus megszorítások esetén az RRT-típusú algoritmusok erősen hagyatkoznak az utófeldolgozásra, simításra. Ez elegendő, ha a tervezett útvonal körül jelentős szabad tér áll rendelkezésre a sima manőver kialakítására. Szűkebb parkolási esetekben ez nem teljesül, így megoldásuk teljesen más megközelítést igényel [11].

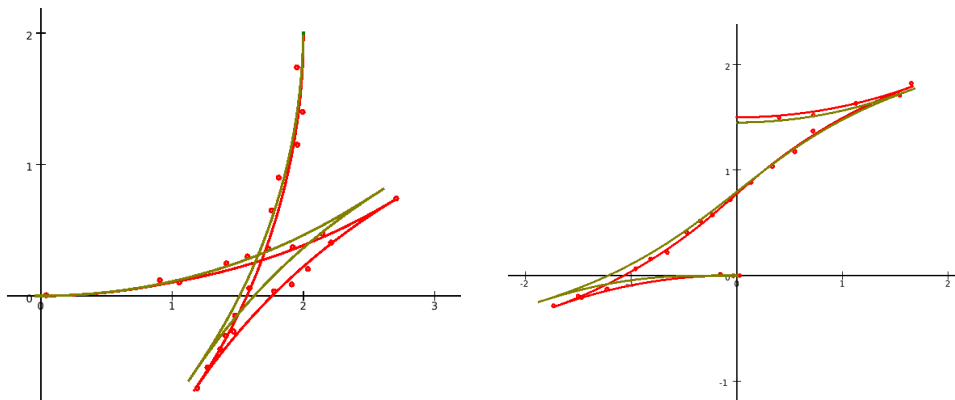
2.6. Sima útvonal létrehozása

Az útvonaltervezés problémája felfogható egy numerikus optimalizációs feladatként is. Ez a megközelítés általában akkor használatos, ha nagy sebesség mellett szeretnénk sima útvonalat tervezni [9]. A globális tervezés problémájának megoldására nem alkalmas, ha több akadályt is ki kell kerülni, ugyanis többszöri újraindítás esetén sem feltétlenül találja meg az optimális útvonalat.

Az optimalizációt érdemes felhasználni az útvonal *lokális simítására* [10]. Ezzel elérhetjük, hogy azokon a helyeken, ahol az útvonal körül van szabad hely, simább manővert alakítsunk ki. Ilyenkor az útvonal hosszabb lesz, viszont a simasága miatt a kontroller dinamikusabban tudja végrehajtani. Az optimalizációban:

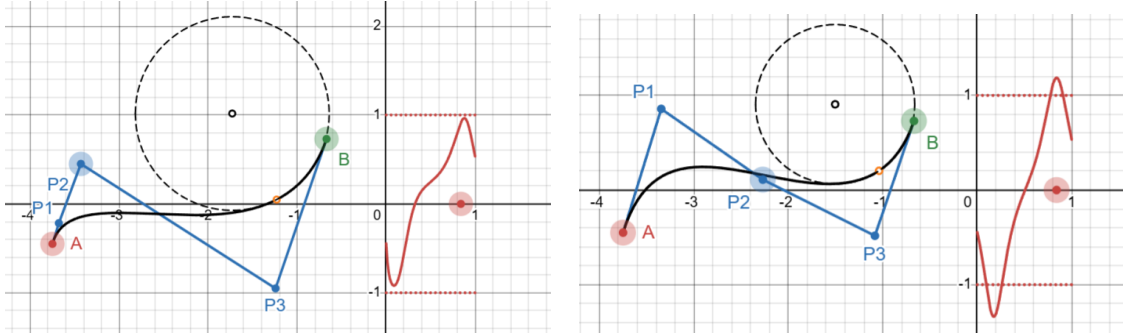
- minimalizáljuk az útvonal hosszát,
- minimalizáljuk a görbületet, és
- maximalizáljuk a távolságot az akadályoktól.

A görbületre és akadálytávolságra vonatkozó esetekben a célfüggvény jobban bünteti a limitek, pontosabban az akadályok és a görbületi korlátok megközelítését. A 2.8. ábrán két útvonal szerepel a simítás előtt és után.



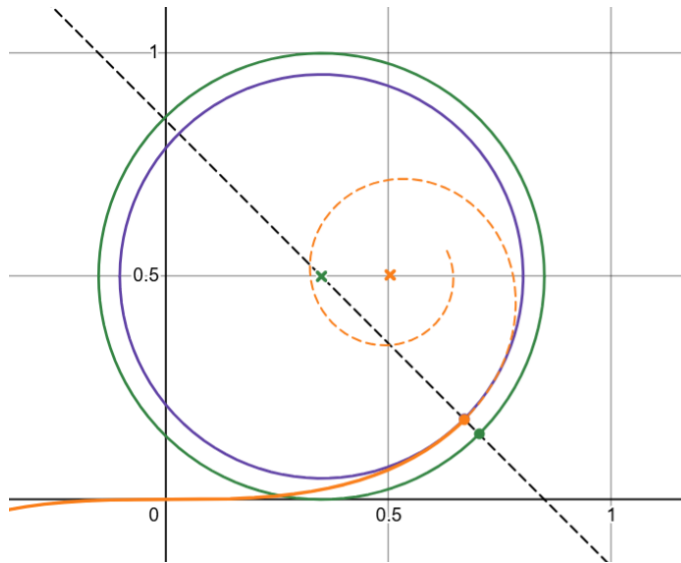
2.8. ábra. Lokális optimalizáció útvonalsimításra. Piros színnel az eredeti, zöldes színnel a simított útvonal látható

A lokális simítás feladatra gyakran használnak *spline* [13] és *Bézier-görbe* [15] alapú módszereket. Ezeknél a legnagyobb kihívás, hogy a görbületi megszorítást nem lehet egyszerűen a kontrollpontokra levetíteni. A végpontokban a folytonosság tetszőleges fokszámig biztosítható a kontrollpontok növelésével, de a görbület felett már nincs ilyen kontroll a teljes görbére nézve. Ezt a problémát mutatja be a 2.9. ábra. Szintén további számítást igényel annak meghatározása, hogy megoldható-e egyáltalán a megkötések által adott útvonal egy adott simító görbével.



2.9. ábra. Egy útvonal simítása Bézier-görbe segítségével. A és B a kezdőpont és a végpont, P -vel a görbe kontrollpontjait jelöltem. A jobb oldalon mindkét ábrán a görbület látható a t időparaméter függvényében. A két ábrán a P_2 kontrollpont pozíciója különbözik. P_1 és P_3 pozíciója a megkötések alapján kiszámítható. $\kappa_{max} = 1$

Ha természetes módon szeretnénk görbületben folytonos útvonalat kapni, akkor érdemes lehet klotoidokat használni [16, 22]. Egy erős kormány szervó motor esetén a jármű hamar eléri a maximális görbületet, és onnantól köríven halad tovább (2.10. ábra). Emiatt a klotoidokat illesztő görbének is lehet használni [21].



2.10. ábra. Egyenes és körív mozgási primitívek görbületfolytonos illesztése klotoiddal

2.7. Geometriai lekérdezések

A járművet és a környezetet általában valamilyen egyszerű geometriával reprezentáljuk, hogy a közöttük lévő metszési, tartalmazási és távolság viszonyok hatékonyan meghatározhatók legyenek. A továbbiakban a jármű kontúrját egy konvex poligonnal, az akadályokat pedig egyenes szakaszokkal közelítjük.

A szenzorfüzió általában háromdimenziós térben detektálja az objektumokat, majd felülnézetben, magassággal címkézett egyenes szakaszokkal reprezentálja őket. Ilyen szakaszokból általában nagyon sok létrejön minden egyes időpillanatban ahogy a szenzorok adatait egyesítjük. Ezeket az adatokat érdemes egyszerűsíteni. Az egyik módszer az egymáshoz közeli pontok összevonása majd az így keletkezett vonalak egyszerűsítése például a Ramer—Douglas—Peucker-algoritmussal [30].

Ahogy az autó halad a térben, folyamatosan újabb területeket fedez fel, melyekről nagy mennyiségű térbeli adatot gyűjt. Az útvonaltervezés közben azonban csak a lokális geometriára van szükségünk. Ezt egy gyors térbeli indexelést [31] végrehajtó adatstruktúrával hatékonyan lekérdezhetjük. Az útvonaltervezőben az R-fa [32] adatstruktúrát használtam, mely rekurzívan felosztja, majd tengelypárhuzamos téglalapokkal veszi körbe a részgeometriákat. Ebben aztán hatékonyan le lehet kérdezni egy tengelypárhuzamos téglalapba eső összes geometriát (akadályt).

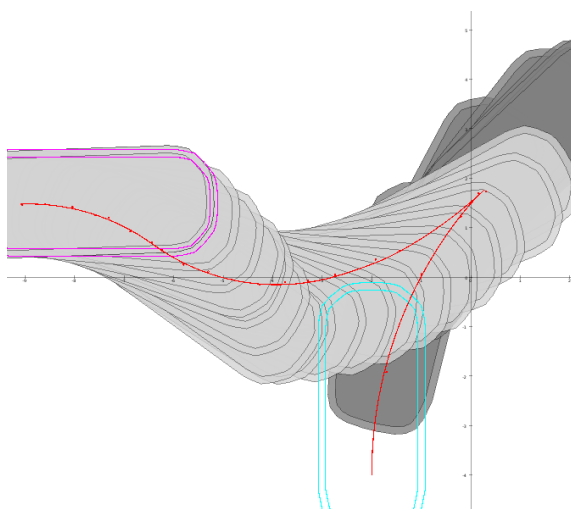
A poligon-reprezentációban egy póz ütközésdetektálása egy konvex poligon és több egyenes szakasz viszonyának meghatározásán múlik. Ehhez használhatunk egyszerű fordulás alapú algoritmusokat (3. fejezet) vagy ismert algoritmusokat mint a söprőegyenesen alapuló Bentley—Ottmann-algoritmus [33].

Mozgási primitívek *általános* ütközésdetektálására két alapvető műveletet használhatunk fel, melyet minden primitívnek támogatnia kell:

1. a primitív kiértékelése diszkrét időpillanatokban,
2. az autó kontúrjának ($\mathcal{K}(\cdot)$) meghatározása az adott pozícióban.

Ezek alkalmazásával egyszerűen implementálható, általános algoritmust kapunk. Sajnos a kontúrok jelentős átfedése miatt az algoritmus nem lesz hatékony (2.11. ábra). A 3. fejezetben bemutatok új, gyorsabb algoritmusokat specifikus primitívekre, melyekkel jelentősen felgyorsíthatók a geometriai lekérdezések.

A lokális optimalizációhoz szükség van a környezet és a jármű távolságára. Ezt általában a poligon oldalainak és a szakaszok távolságára vezetjük vissza. Erre az esetre is bemutatok hatékonyabb algoritmusokat a 3. fejezetben.



2.11. ábra. Ütközésdetektálás az útvonal mintavételezésével

3. fejezet

Megvalósítás és alkalmazás

3.1. Heurisztikák

Az A* algoritmusnál admisszibilis és konzisztens heurisztikával tudjuk garantálni az optimalitást. Ebben a részben bemutatok néhány gyakran használt heurisztikát az előnyeikkel és hátrányaikkal együtt.

3.1.1. Egyszerű heurisztikák

Talán a legegyszerűbb heurisztika az Euklideszi távolság.

$$h_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Ennek előnye, hogy gyorsan számítható, így kis költséggel lecsökkenthetjük a keresési teret a Dijkstra-algortmushoz képest. (Ha az A*-ban $h = 0$, akkor a Dijkstra-algoritmust kapjuk.) Pontszerű robotoknál működik jól. Ha vannak nemholonomikus korlátok is, akkor gyakran félrevezeti a keresést.

A Manhattan távolság a koordináták végtelen normája.

$$h_M = |x_1 - x_2| + |y_1 - y_2|$$

Hasonló előnyei és hátrányai vannak, mint az Euklideszi heurisztikának.

Az előbbi heurisztikák kiegészíthetők az orientációk közti távolsággal. Ezzel figyelembe vehetjük, hogy egy jármű orientációja is számít, illetve hogy megfordulni egy költséges manőver még a szabadtéri esetben is. Legyen szögtáv(θ_1, θ_2) a legkisebb abszolút értékű δ szám, melyre $\theta_1 + \delta \approx \theta_2$. Itt két szög ekvivalens, ha 2π radián többszörösével

térnek el egymástól. A kiegészített heurisztikák:

$$h_{E+} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + r_{min}^2 \cdot \text{szögtáv}(\theta_1, \theta_2)^2},$$

$$h_{M+} = |x_1 - x_2| + |y_1 - y_2| + r_{min} \cdot |\text{szögtáv}(\theta_1, \theta_2)|.$$

Itt a súlyozás értelme, hogy a jármű θ szögű elfordításához legalább $r_{min}\theta$ ívhosszt kell megtenni. Annak ellenére, hogy a gyakorlatban jól működnek, a kiegészített heurisztikák-
kal vigyázni kell, mert nem teljesül rájuk a háromszög-egyenlőtlenség, így nem metrikák.

3.1.2. Nemholonomikus heurisztikák

Kis távolságok esetén a hátralévő távolságot általában a nemholonomikus megszo-
rításokat reprezentáló heurisztikák becslik legjobban. A szabadtéri parkolási esetekben
a Reeds–Shepp-heurisztika optimális, azaz pontosan a hátralévő távolságot adja. Annak
ellenére, hogy az algoritmus futási ideje $O(1)$, sokkal költségesebb az egyszerű heurisztika-
kánál, így néhány cikkben közelítéseket alkalmaznak.

A magas számítási költség egyik forrása, hogy bár az eredeti Reeds–Shepp cikkben
[4] jelentősen csökkentették a kipróbálandó esetek számát, továbbra is bent maradt 42 ki-
próbálandó konfiguráció. Ezek közül nem mindegyik fordul elő az optimális megoldások
között, és bizonyos útvonalak csak a bemeneti paraméterek szűk intervallumára lesznek
optimálisak (A.11 ábra). A paraméterek osztályozásával tehát jelentősen csökkenthető
a kipróbálandó esetek száma. A kutatás megkezdése óta megjelent egy cikk [34], mely
pontosan ezt a problémát oldja meg, így a dolgozatban részletesebben nem tárgyalom.

A Reeds–Shepp algoritmus azért is drága, mert minden eset leellenőrzéséhez trigo-
nometrikus függvényeket, inverz trigonometrikus függvényeket és polárkoordináta kon-
verziót kell számítanunk [35]. Ezek transzcendens függvények, melyeket a számítógépen
csak numerikus közelítéssel tudunk kiszámítani. A legdrágábbak az inverz trigonometri-
kus függvények számítása. Itt általában a valamennyi numerikus pontosságot feláldozha-
tunk, hogy sokkal gyorsabb közelítést kapjunk [36].

Az `atan2()` függvény esetén egy [36]-nál sokkal pontosabb, de az `std::atan2f()` imple-
mentációnál még mindig 50-szer gyorsabb módszer a függvény polinomiális közelítése
[37] (3.1. forráskód).

A közelítés gyors, mivel a konstansok kis helyen elférnek a cache-ben, és a függvény
a streaming processzoron elágazás nélkül le tud futni. Ez a függvény csak a $[-1, 1]$ in-

```

inline float atan_approximation(float x) {
    float a1 = 0.99997726f;
    float a3 = -0.33262347f;
    float a5 = 0.19354346f;
    float a7 = -0.11643287f;
    float a9 = 0.05265332f;
    float a11 = -0.01172120f;

    float x_sq = x*x;
    return x * (a1 + x_sq * (a3 + x_sq * (a5 + x_sq * (a7 +
        x_sq * (a9 + x_sq * a11)))));
}

```

3.1. forráskód. atan() függvény polinomiális közelítése

```

float atan2(const float y, const float x) {
    // Ensure input is in [-1,1]
    bool swap = fabs(x) < fabs(y);
    float atan_input = (swap ? x : y) / (swap ? y : x);

    // Approximate atan
    float res = atan_approximation(atan_input);

    // If swapped, adjust atan output
    res = swap ? (atan_input >= 0 ? PI : -PI)/2 - res : res;
    // Adjust quadrants
    if (x < 0 && y >= 0) { res += PI; } // 2nd
    else if (x < 0 && y < 0) { res += -PI; } // 3rd

    return res;
}

```

3.2. forráskód. atan2() függvény gyors közelítése.

tervállumon jó közelítés. Egy helyes atan2() függvényhez le kell kezelni több esetet (3.2. forráskód).

Az arcsin() és arccos() függvények egymásból származtathatóak, így elég egyiket implementálni. Hochstrasser szerint érdemes a megoldást $P(x) \cdot \sqrt{1-x}$ alakban keresni, ahol $P(x)$ egy polinom [38]. Egy konkrét implementáció (3.3. forráskód) az Nvidia oldalán [39] elérhető.

3.1.3. Objektumtudatos heurisztikák

Az eddig bemutatott heurisztikák szabadtéri esetben nagyon jól működnek, azonban nem vezetik át a járművet egy akadályokkal teli környezeten. A legtöbb objektumtudatos heurisztika célja, hogy az akadályokat messziről elkerülje. Ezen heurisztikáktól azt várjuk, hogy a jármű át tudjon hajtani két kapuoszlop között vagy be tudjon parkolni

```
float asin(float x) {  
    float negate = float(x < 0);  
    x = abs(x);  
    float ret = -0.0187293;  
    ret *= x;  
    ret += 0.0742610;  
    ret *= x;  
    ret -= 0.2121144;  
    ret *= x;  
    ret += 1.5707288;  
    ret = 3.14159265358979*0.5 - sqrt(1.0 - x)*ret;  
    return ret - 2 * negate * ret;  
}
```

3.3. forráskód. arcsin() függvény gyors közelítése.

merőlegesen két autó közé középre.

Erre a feladatra gyakran használt módszerek a *mező alapú heurisztikák* [9, 10]. Ennél az ismert akadályok köré egy potenciálmezőt definiálunk, mely az akadályoktól vett távolság szerint monoton csökken. A heurisztikához hozzáadjuk a potenciálmező értékét, így motiválva a szabad tér felfedezését.

Egy másik módszer szerint *szélességi keresést* futtatunk egy diszkretizált, kis felbontású állapot térben. Ez megad egy hozzávetőleges útvonalat a térkép bármely pontjából a célba. A szélességi keresés komplexitásában közelítően ekvivalens az eredeti problémával, így csak kis felbontással alkalmazható.

A módszerek legfőbb limitációja a teljesítmény. A potenciálmező online kiértékelése az objektumok száma miatt költséges, előszámítása pedig nagy felbontás esetén szintén lassú lehet. Ha csak alacsony felbontáson dolgozunk, akkor előfordulhat, hogy szűk átjárókat nem fogunk megtalálni. Minden ilyen megoldás problémája, hogy szűk parkolási esetekben a potenciálmező gyakorlatilag konstans, mivel a manőver során végig nagyon kevés hely van a jármű körül.

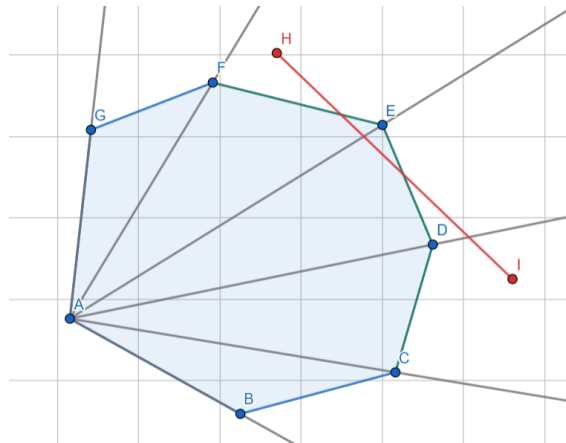
3.2. Geometriai lekérdezések

Korábban láthattuk, hogy a mintavételezésen alapuló módszer költséges a minták átfedése miatt. Az útvonaltervezés során nagyon specifikus geometriai lekérdezéseket kell megválaszolnunk: A jármű geometriája konvex, az akadályok egyenes szakaszok és ha ütközést detektálunk, akkor a többi metszés ténye és helye irreleváns. A következőkben bemutatok több, erre az esetre optimalizált ütközésdetektálási algoritmust.

3.2.1. Analitikus ütközésetektálás

Az ütközésetektálás kezdetén feltehetjük, hogy a jármű ütközésmetas állapotban van, hiszen az útvonaltervező elérte az adott pozíciót. Ezután le kell ellenőriznünk, hogy a célkonfiguráció is ütközésmetas-e. Ha nem, akkor a célkonfigurációt le kell zárni, hogy később ne próbáljuk megint elérni.

Ehhez egy pöz ütközésetektálását kell elvégeznünk. Itt először leellenőrizzük, hogy egyik szakaszvégpont sincs benne a poligonban. Ez megtehető $O(Q \cdot \log N)$ időben [40], ahol N a poligon csúcsszáma és Q a pontok száma. Emlékezzünk, hogy a szakaszok számának csökkentésére R-fát [32] használunk. Ha így nem találtunk ütközést, akkor minden szakaszvégpont a poligonon kívül van. Ezután elég a poligon azon oldalait ellenőrizni, melyek a két szakaszvégpont között vannak a polárszöges rendezésben (3.1. ábra). A konvexitás miatt ezen az intervallumon szintén kereshetünk binárisan, tehát az algoritmus összköltsége $O(Q \cdot \log N)$. Ez sokkal jobb, mint a naiv $O(Q \cdot N)$ idejű megoldás.

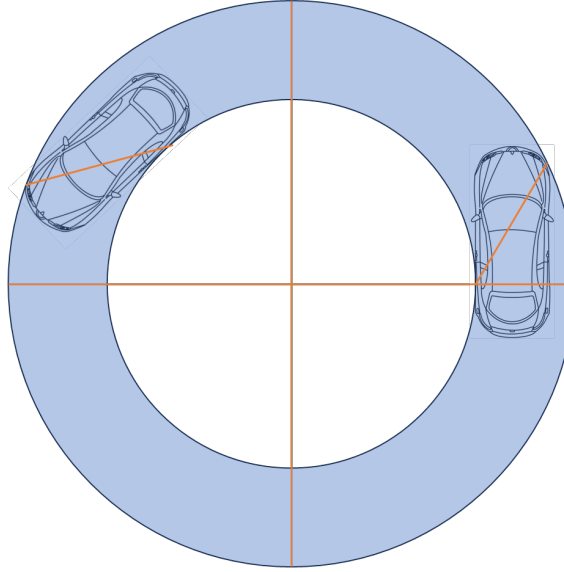


3.1. ábra. Poligon-szakasz ütközésetektálás $O(\log N)$ időben

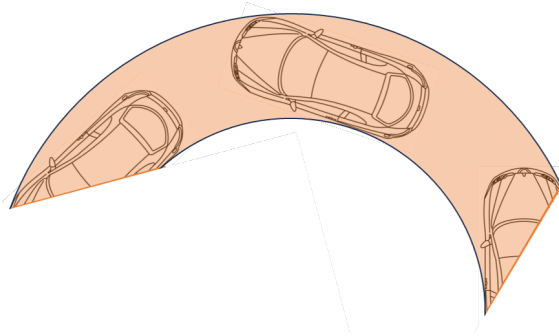
Ezután tudjuk, hogy a mozgási primitív kezdő- és célkonfigurációja ütközésmetas. Ezek azok a részei a primitív által lefedett területnek, melyek a jármű kontúrját követik. A köztes rész a primitív geometriáját követi. Egy egyenes mozgási primitív esetében a köztes rész egyszerűen egy téglalap, melynek hossza a mozgási primitív hossza, szélessége pedig a jármű kontúr maximális szélessége. Ezen téglalap ütközése $O(Q)$ időben ellenőrizhető, ahol Q a szakaszok száma.

Egy körív esetén a külső és belső ívek sugarának és kezdetének meghatározása átgondolást igényel (3.2. ábra). A belső ív sugara a kontúr legközelebbi pontja lesz a kanyarodás középpontjához képest. A külső ív sugara viszont nem az ezzel szemben lévő pont, hanem a kanyarodási középponttól legtávolabbi, általában a jármű orránál. Ebből

kifolyólag a külső és belső ívek nem ugyanott kezdődnek, így a keletkezett alakzatra már a ponttartalmazás problémája sem egyértelmű (3.3. ábra).



3.2. ábra. Egy körív által lefedett munkaterület



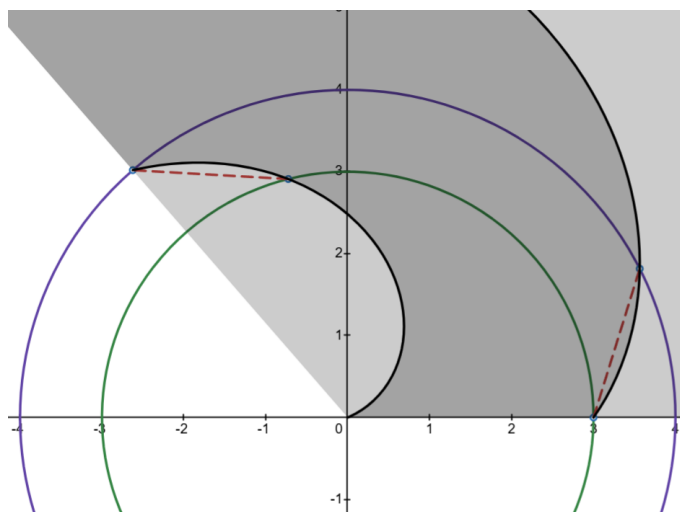
3.3. ábra. Egy körív által lefedett munkaterület köztes része

A ponttartalmazásra emiatt először alkalmazunk egy távolságfüggő forgatás transzformációt, mely a két ív kezdőpontjait és a kanyarodás középpontját, az origót egy egyenesre transzformálja. A forgatás szöge

$$\rho = \theta + \alpha \frac{r - r_1}{r_2 - r_1},$$

ahol θ az ív kezdőszöge, α a két ív kezdőszöge közötti különbség (a jármű paramétereiből számítható konstans), r_1 és r_2 pedig a belső és a külső ív sugara. A forgatás által egy egyenesre eső pontokat a 3.4. ábra szemlélteti.

Az így keletkezett alakzat teljesen lefedi a köztes ellenőrizendő területet, mert tudjuk, hogy a primitív kezdő- és célkonfigurációja ütközésmentes. Ha egy pont eredetileg a köztes ütközőzónában volt, akkor a transzformáció után is benne lesz. Ha egy pont a köztes



3.4. ábra. Távfolságfüggő forgatás transzformáció

ütközőzónán és a végpontok kontúrján is kívül volt, akkor a transzformáció miatt nem kerülhet bele, mert a forgatás a sugarat nem változtatja.

A transzformáció után egy pont akkor tartalmazott, ha $r_1 \leq r \leq r_2$ és $\theta_1 \leq \theta \leq \theta_2$. Az imént leírt transzformációkat és összehasonlításokat természetes lehet polárkoordináta konverzióval implementálni, azonban a mérésekből kiderül, hogy érdemes ezt inkább elkerülni. A szögek összehasonlítása elvégezhető a vektorok vektoriális szorzatának kiszámításával, a forgatás pedig egy forgatás mátrixszal.

Ha eddig nem detektáltunk ütközést, akkor az összes szakaszvégpont az ütközőzónán kívül van. Ezután csak akkor lehetséges ütközés, ha a szakasz metszi a külső körívet. (A végpontok ütközésmentesek, illetve a belső körív konvexitása miatt nem vehet részt ütközésben.) Ezután már „csak” egy körív-szakasz ütközést kell megoldani. Itt mindenképp figyelni kell a másodfokú egyenlet gyökeinek numerikusan stabil meghatározására [41].

4. fejezet

Szimulációs eredmények

Az autóiparban használt elméleti módszereknek és az elkészült szoftvernek éles használat előtt részletes szimulációs teszteken kell megfelelnie. Ezek célja részben a módszer validálása, másfelől pedig a paraméterek finomhangolása a biztonság és a vezetési élmény érdekében.

4.1. Szabványok

A szoftvert kettő, parkolási eseteket tartalmazó szabvány esetein teszteltem. A parkolási esetek az A. függelékben ábrán is megtekinthetők. Az egyik szabvány az ISO 20900:2023 [42]. A szabvány a merőleges, párhuzamos és egy garázsba parkolás esetet mutat be részletesen (A.1., A.2., A.3. ábrák). Ezt kell az önvezető autónak különböző kiindulási és célkonfigurációkon teljesíteni.

Az iVista [43] egy kínai szabvány, mely önvezető autók autonóm funkcióit sorolja kategóriákba a teljesítményük szerint. A szabvány által definiált parkolási esetek az ISO 20900 szellős eseteihez képest meglehetősen szűkek. A manőverek teljesítéséhez több esetben közel kell menni akadályokhoz, mely már a szenzorfüzió számára is kihívást jelentő feladat (A.4., A.5., A.6. ábrák). A két szabvány fontosabb paramétereit hasonlítja össze a 4.1. táblázat.

Szabvány	ISO 20900	iVista
Merőleges parkolóhely szélessége	W+1,2m	W+0,8m
Szemközti akadály távolsága merőlegesnél	7m	5,5m
Párhuzamos parkolóhely hossza	L·1,25	L·1,15
Szemközti akadály távolsága párhuzamosnál	4,5m	3,5m

4.1. táblázat. ISO 20900:2023 és iVista szabványok összehasonlítása. (W a jármű szélessége, L pedig a hosszúsága)

4.2. Tesztelés módja, metrikák

A teszteléshez nagy számú (több, mint 5000) véletlenszerű tesztet generáltam. A tesztek a korábban tárgyalt parkolási esetekből kerültek ki. A kiértékelés az alábbi metrikák szerint történt:

1. Rendelkezésre állás: megmutatja, hogy a tesztek mekkora részét tudja megoldani a tervező egy parkolási eseten belül.
2. Útvonal hossza: a megoldott tesztekre az útvonal átlagos hossza méterben.
3. Manőverek száma: a megoldott tesztekre az irányváltások átlagos száma.
4. Iterációs szám: a megoldáshoz szükséges csúcskiterjesztések száma.
5. Futási idő: a tervező átlagos futási ideje a parkolási eset összes tesztjére.
6. Oldalirányú rándulás metrika (Lateral jerk metric [44]): az útvonal simaságát méri. A simítási eljárás hatékonyságát mutatja.

4.2.1. Hiperparaméter beállítás optimalizációval

A tesztek lefuttatása sok időt (30+ perc) vett igénybe, így a hiperparaméterek beállítására egy sztochasztikus gradiens módszert, az AdaMax-ot [45] használtam. A hiperparaméterek közt szerepel:

1. a heurisztikák relatív súlya;
2. a heurisztikák összsúlya;
3. az irányváltás relatív költsége;
4. a rács felbontása az (x, y, θ) dimenziók mentén;
5. a simításhoz használt numerikus módszer paraméterei és a költségfüggvény súlyai;
6. a mozgási primitívek hosszai.

4.3. Eredmények

A kutatási kérdések között szerepelt a *rács optimális paraméterezése*. Itt fontos eredmény, hogy az esetek nagy részében a tervező rendelkezésre állása és az útvonal hossza nem romlik, ha a rács méretét a legkisebb primitív méretéig, nagyjából 10 cm-ig, szögfelbontásban pedig akár 15° -ig növeljük. A nagyon szűk párhuzamos és blokkolt mérőleges parkolási esetekben szükség lehet nagyobb szög szerinti felbontásra, mert ott a limitált longitudinális hely miatt a járművet csak apró elfordulásokkal lehet manőverezni. Ilyenkor érdemes lehet a szűk helyet előre szűrni, és alapból nagyobb szög szerinti felbontást alkalmazni. Ennek vizsgálata további kutatást igényel.

A *mozgási primitívek optimális konfigurációjára* nem kaptam egyértelmű választ, mert több konfiguráció is hasonlóan jól teljesített. A jól teljesítő konfigurációkra általában elmondható, hogy

- érdemes rövidebb és hosszabb primitíveket is szerepeltetni.
- A legrövidebb primitív a rács méretéhez igazodik.
- A primitívek hosszai nem osztják egymást. (Például az 1:2:4 arányú hosszok nem olyan jók, mint az 1:3:7 arányúak.)

A hosszabb primitívek tesztelése csak az új, analitikus ütközésetektálás alkalmazása esetén érdemes. A mintavételezéses megoldás esetén ugyanis egy n -szer hosszabb primitív ütközésetektálása n -szer költségesebb is, így mindig érdemesebb a hosszabb primitívet kis részletekben leellenőrizni. A hosszabb primitívek jelentősen csökkentik az iterációszámot, ha nagyobb távot kell megtenni.

Az *objektumtudatos heurisztikák* az én alkalmazási területemen nem teljesítettek jól. A blokkolt, utolsó helyes parkolási esetben nem csökkentette az iterációszámot, ellenben a heurisztikában szükséges geometriai lekérdezések jelentősen megnövelték a futási időt.

A 4.2. táblázatban összefoglalom a tervező javulását a rendelkezésre állás metrika szerint. A tervező a táblázat alapján jelentős javulást mutat az etalonnak vett Hibrid A* tervezőhöz képest.

A módszereket a Robert Bosch Kft. OneParking platformjának útvonaltervezőjébe implementáltam. Az új funkciók nagy része túljutott a prototípus fázison, és szoftveres, majd autós teszteken is helyt állt.

	Hibrid A*	Saját tervező
ISO merőleges	100%	100%
ISO párhuzamos	100%	100%
ISO garázs	22%	86%
iVista merőleges	100%	100%
iVista párhuzamos	90%	95%
iVista átlós	72%	84%
utolsó előtti hely	31%	79%
utolsó hely	0%	90%

4.2. táblázat. A tervező rendelkezésre állásának javulása a parkolási eset függvényében. A bal oldalon a Hibrid A* tervező szerepel alapbeállításokkal, a jobb oldalon pedig a saját tervező az optimalizált paraméterekkel és implementációval.

5. fejezet

Összefoglalás és eredmények

A dolgozatban bemutatam az útvonaltervezéssel kapcsolatos legfontosabb fogalmakat és módszereket. Széles körű áttekintést adtam a nemholonomikus járművek útvonaltervezésénél használt legfontosabb módszerekről. Bemutattam egy, a tudomány jelen állása szerinti legjobb útvonaltervezőt. Új algoritmusokat és módszereket javasoltam a tervező költséges részeinek felgyorsítására. A tervező hiperparamétereit aztán önvezető autók parkolására optimalizáltam numerikus optimalizáció segítségével.

A dolgozat új módszert javasol geometriai lekérdezések hatékony megválaszolására és meglévő módszereket kombinál a heurisztika számítás felgyorsítására. A dolgozat útmutatást nyújt a területet alkalmazóknak a paraméterek beállítására, és egy újrafelhasználható keretrendszert biztosít azok finomhangolására.

A módszereket a Robert Bosch Kft. OneParking platformjának útvonaltervezőjébe implementáltam. Az új funkciók nagy része túljutott a prototípus fázison, és szoftveres, majd autós teszteken is helyt állt.

5.1. Nyilatkozat a használt eszközökről

A dolgozat elkészítéséhez az Overleaf¹ L^AT_EX szerkesztőt használtam. A dolgozat szövegének megírásához nagy nyelvi modellt (mesterséges intelligenciát) *nem* használtam. Az ábrákat a Desmos², GeoGebra³ és Matlab⁴ eszközökkel készítettem. A szimulációk megjelenítésére a Qt⁵ keretrendszert használtam.

¹ www.overleaf.com

² www.desmos.com

³ www.geogebra.org

⁴ www.matlab.mathworks.com

⁵ www.qt.io

Köszönetnyilvánítás

A kutatás az Egyetemi Kutatói Ösztöndíj Program, a Kulturális és Innovációs Minisztérium, valamint a Nemzeti Kutatási, Fejlesztési és Innovációs Alap támogatásával készült. Ezúton is szeretnénk köszönetet mondani a magyarországi Bosch csoportnak, hogy a tanulmány elkészítéséhez szükséges kutatás során a vállalat szakmai támogatást nyújtott.

Köszönöm a szakmai útmutatást Dr. Lócsi Leventének és Mócsai Tamásnak.



ELTE



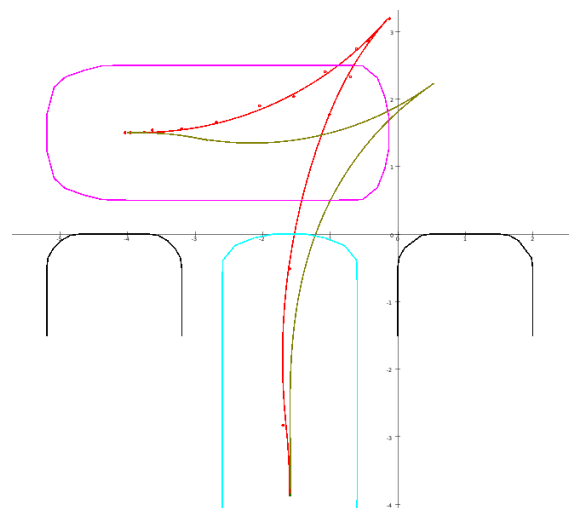
BOSCH

5.1. ábra. Támogatók

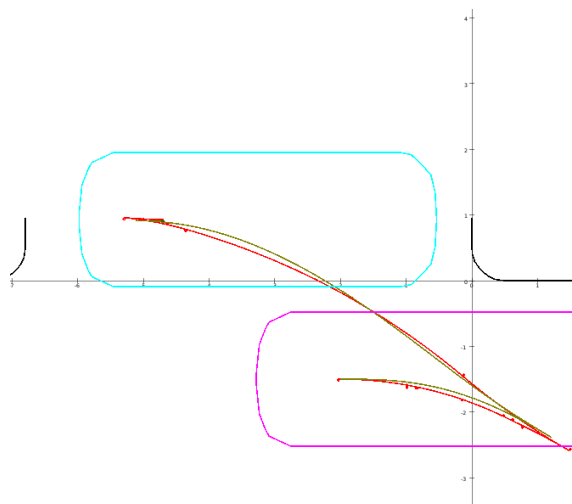
A. függelék

Függelék

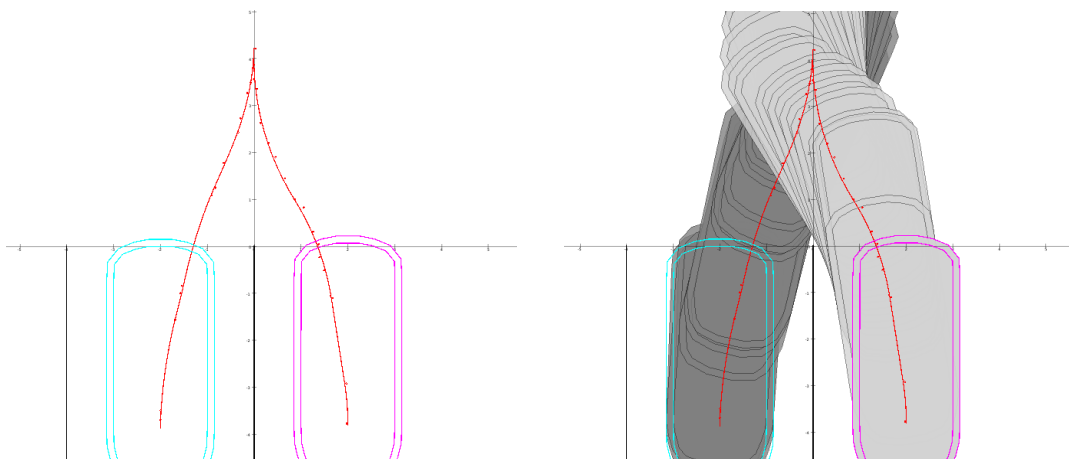
A.1. Parkolási esetek



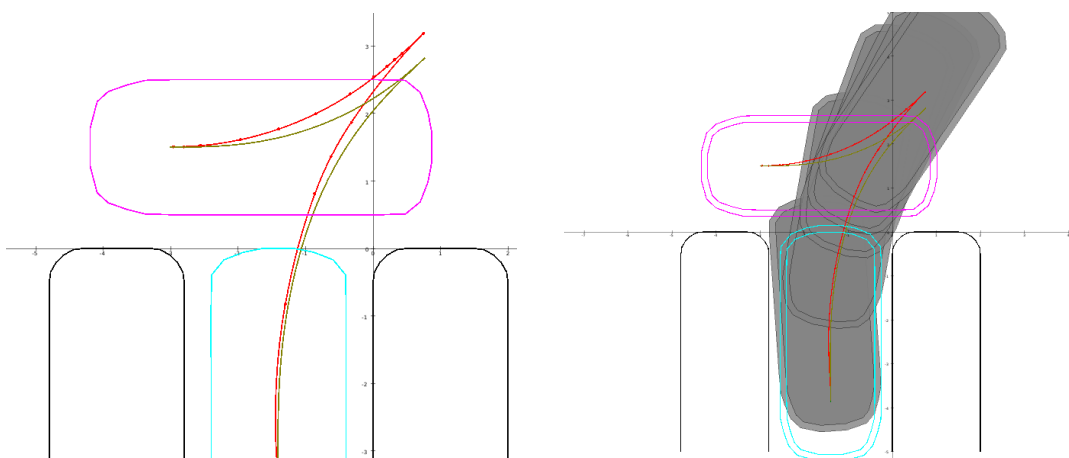
A.1. ábra. ISO 20900:2023 mérőleges parkolás eset



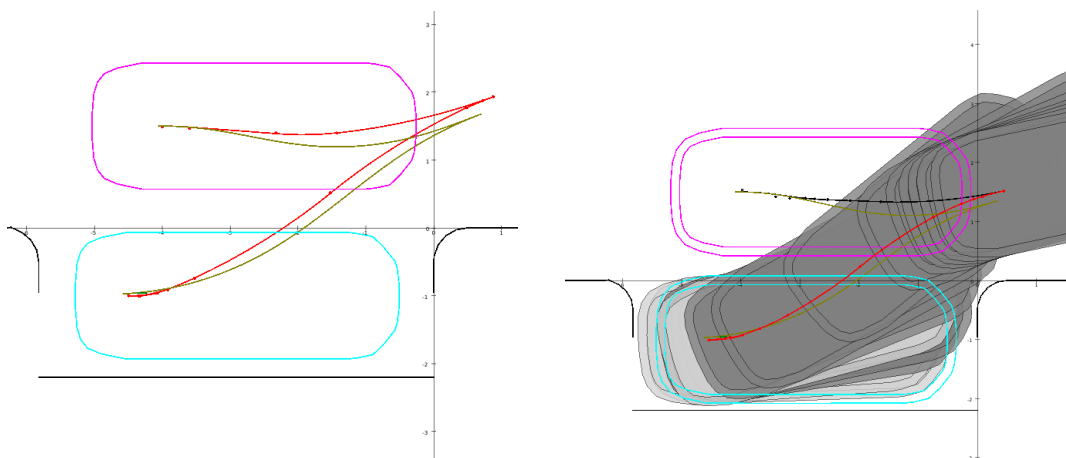
A.2. ábra. ISO 20900:2023 párhuzamos parkolás eset



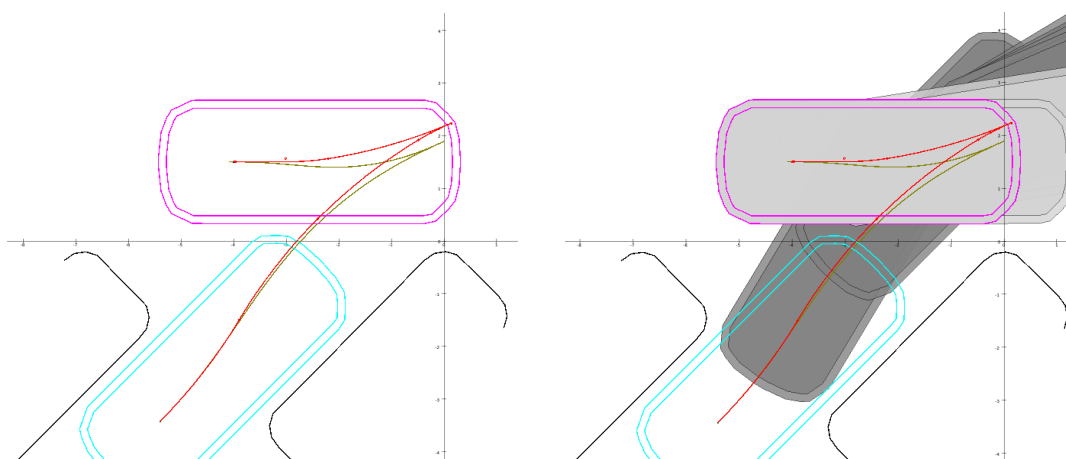
A.3. ábra. ISO 20900:2023 garázsba parkolás eset



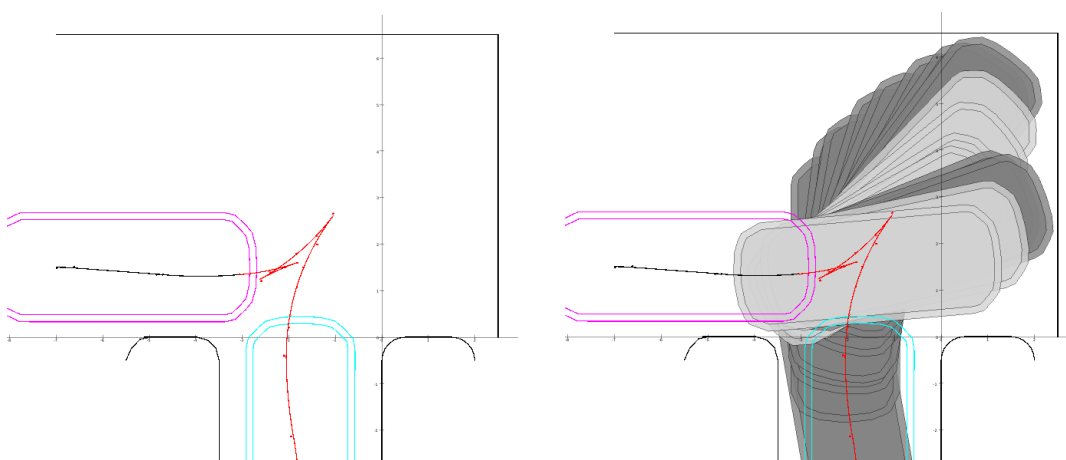
A.4. ábra. iVista merőleges parkolás eset



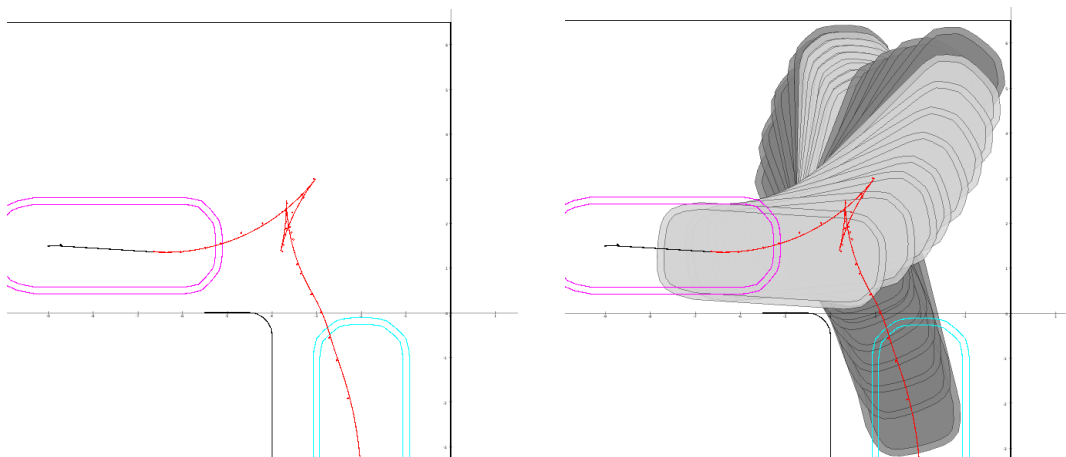
A.5. ábra. iVista párhuzamos parkolás eset



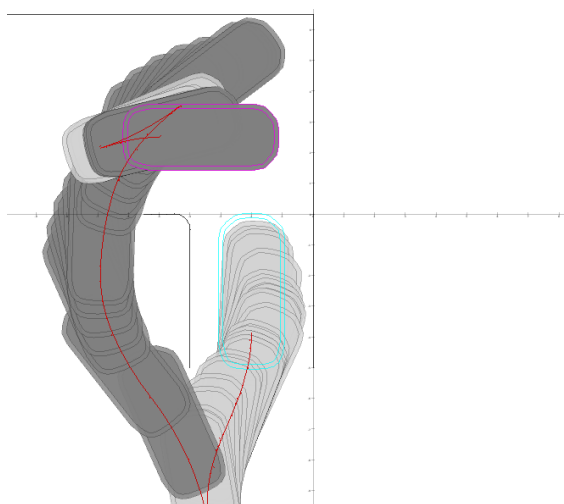
A.6. ábra. iVista átlós parkolás eset



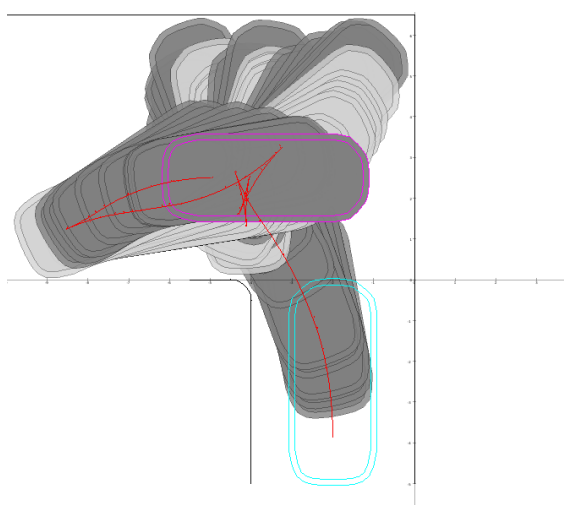
A.7. ábra. Utolsó előtti helyre parkolás esete egy parkolóházban



A.8. ábra. Utolsó helyre parkolás esete egy parkolóházban

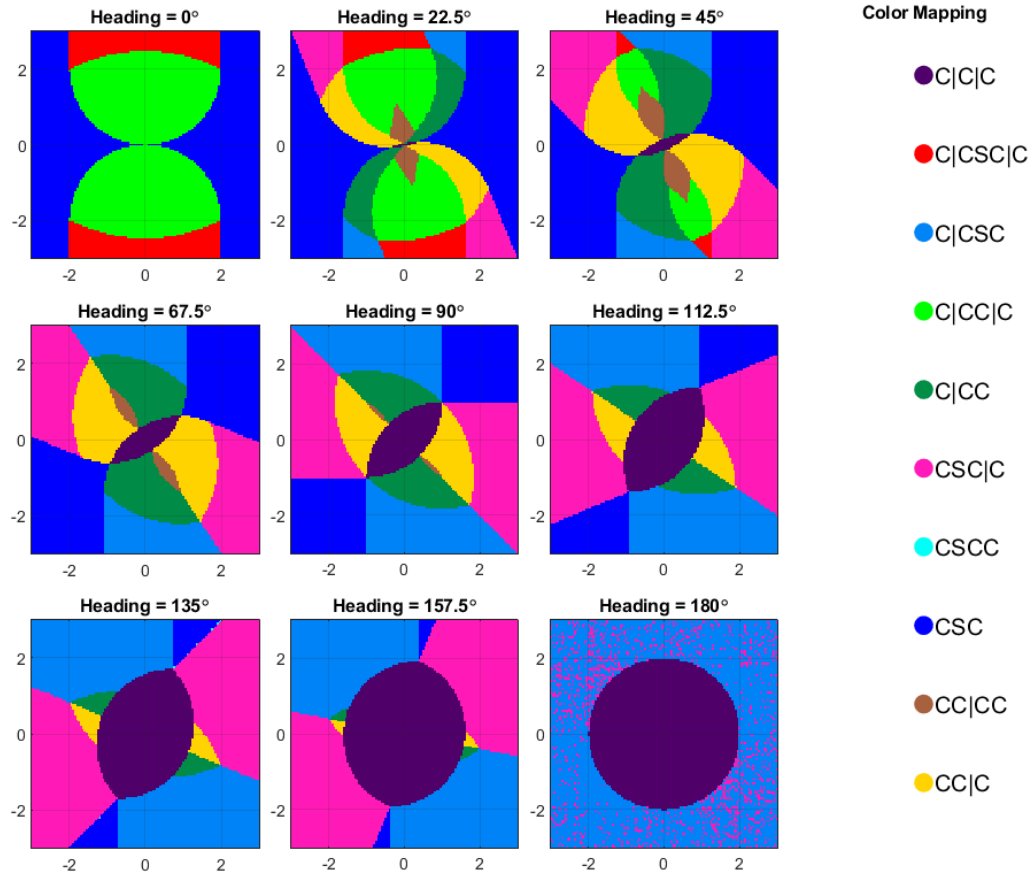


A.9. ábra. Utolsó helyre parkolás kreatív megoldása aluldefiniált esetben

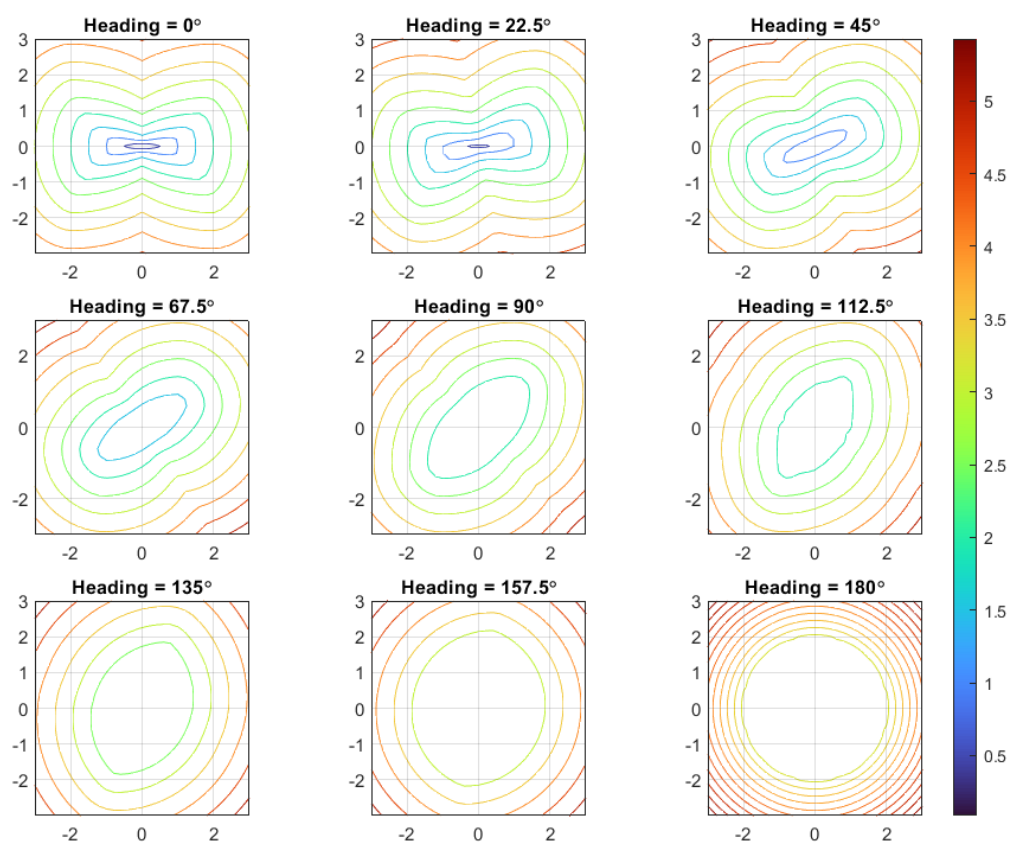


A.10. ábra. Utolsó helyre parkolás esete egy nagyobb járművel

A.2. Reeds–Shepp-útvonalak



A.11. ábra. Reeds–Shepp-útvonalak osztályozása a primitívek szerint. A jármű a $(0,0,0)$ pontból indul és az $(x,y,heading)$ pontba érkezik. A primitívek jelölése az eredeti cikk [4] szerint. Az ábrán szereplő határvonal menti hibák numerikus pontatlanságokból és a probléma aluldefiniáltságából adódik.



A.12. ábra. Reeds–Shepp-útvonalak hossza. A jármű a $(0, 0, 0)$ pontból indul és az $(x, y, \text{heading})$ pontba érkezik

Irodalomjegyzék

- [1] Jürgen Ackermann. „Robust car steering by yaw rate control”. *29th IEEE conference on decision and control*. IEEE. 1990, 2033–2034. old.
- [2] Philip Polack és tsai. „The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?”: *2017 IEEE intelligent vehicles symposium (IV)*. IEEE. 2017, 812–818. old.
- [3] Lester E Dubins. „On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents”. *American Journal of mathematics* 79.3 (1957), 497–516. old.
- [4] James Reeds és Lawrence Shepp. „Optimal paths for a car that goes both forwards and backwards”. *Pacific journal of mathematics* 145.2 (1990), 367–393. old.
- [5] De Jong Yeong és tsai. „Sensor and sensor fusion technology in autonomous vehicles: A review”. *Sensors* 21.6 (2021), 2140. old.
- [6] Franco P Preparata és Michael I Shamos. *Computational geometry: an introduction*. Springer Science & Business Media, 2012.
- [7] Georg Tanzmeister és tsai. „Efficient evaluation of collisions and costs on grid maps for autonomous vehicle motion planning”. *IEEE Transactions on Intelligent Transportation Systems* 15.5 (2014), 2249–2260. old.
- [8] Julius Ziegler és Christoph Stiller. „Fast collision checking for intelligent vehicle motion planning”. *2010 IEEE intelligent vehicles symposium*. IEEE. 2010, 518–522. old.
- [9] David González és tsai. „A review of motion planning techniques for automated vehicles”. *IEEE Transactions on intelligent transportation systems* 17.4 (2015), 1135–1145. old.
- [10] Dmitri Dolgov és tsai. „Practical search techniques in path planning for autonomous driving”. *ann arbor* 1001.48105 (2008), 18–80. old.

- [11] Iram Noreen, Amna Khan és Zulfikar Habib. „Optimal path planning using RRT* based approaches: a survey and future directions”. *International Journal of Advanced Computer Science and Applications* 7.11 (2016).
- [12] Ben Zhang és Denglin Zhu. „A new method on motion planning for mobile robots using jump point search and Bezier curves”. *International Journal of Advanced Robotic Systems* 18.4 (2021), 17298814211019220. old.
- [13] Tomas Berglund és tsai. „Planning smooth and obstacle-avoiding B-spline paths for autonomous mining vehicles”. *IEEE transactions on automation science and engineering* 7.1 (2009), 167–172. old.
- [14] Takashi Maekawa és tsai. „Curvature continuous path generation for autonomous vehicle using B-spline curves”. *Computer-Aided Design* 42.4 (2010), 350–359. old.
- [15] Reinis Cimurs, Jaepyung Hwang és Il Hong Suh. „Bezier curve-based smoothing for path planner with curvature constraint”. *2017 First IEEE International Conference on Robotic Computing (IRC)*. IEEE. 2017, 241–248. old.
- [16] Mišel Brezak és Ivan Petrović. „Real-time approximation of clothoids with bounded error for path planning applications”. *IEEE Transactions on Robotics* 30.2 (2013), 507–515. old.
- [17] Wei Liu és tsai. „A systematic survey of control techniques and applications in connected and automated vehicles”. *IEEE Internet of Things Journal* 10.24 (2023), 21892–21916. old.
- [18] Sh Kh Soltakhanov, Mikhail Yushkov és S Zegzhda. *Mechanics of non-holonomic systems: A new class of control systems*. Springer Science & Business Media, 2009.
- [19] Francesco Bernardini és Aaron T. Becker. *Shortest Path for Forward and Reverse Motion of a Car*. URL: [demonstrations . wolfram . com / ShortestPathForForwardAndReverseMotionOfACar/](https://demonstrations.wolfram.com/ShortestPathForForwardAndReverseMotionOfACar/) (elérés dátuma 2025. 04. 26.).
- [20] Holger Banzhaf és tsai. „From G 2 to G 3 continuity: Continuous curvature rate steering functions for sampling-based nonholonomic motion planning”. *2018 IEEE Intelligent Vehicles Symposium (IV)*. IEEE. 2018, 326–333. old.

- [21] Holger Banzhaf és tsai. „Hybrid curvature steer: A novel extend function for sampling-based nonholonomic motion planning in tight environments”. *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE. 2017, 1–8. old.
- [22] Holger Banzhaf. „Nonholonomic Motion Planning for Automated Vehicles in Dense Scenarios”. Dissz. Karlsruhe, Karlsruher Institut für Technologie (KIT), 2020.
- [23] Pulkit Paliwal. „A survey of a-star algorithm family for motion planning of autonomous vehicles”. *2023 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECs)*. IEEE. 2023, 1–6. old.
- [24] Wenrong Jiang. „Analysis of iterative deepening a* algorithm”. *IOP Conference Series: Earth and Environmental Science*. 693. köt. 1. IOP Publishing. 2021, 12028. old.
- [25] Alexander Reinefeld és T. Anthony Marsland. „Enhanced iterative-deepening search”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 16.7 (2002), 701–710. old.
- [26] Adel Ammar. „ERA*: Enhanced Relaxed A* algorithm for solving the shortest path problem in regular grid maps”. *Information Sciences* 657 (2024), 120000. old.
- [27] Taeg-Keun Whangbo. „Efficient modified bidirectional A* algorithm for optimal route-finding”. *New Trends in Applied Artificial Intelligence: 20th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2007, Kyoto, Japan, June 26-29, 2007. Proceedings* 20. Springer. 2007, 344–353. old.
- [28] Sven Koenig és Maxim Likhachev. „Fast replanning for navigation in unknown terrain”. *IEEE transactions on robotics* 21.3 (2005), 354–363. old.
- [29] Jonathan D Gammell, Siddhartha S Srinivasa és Timothy D Barfoot. „Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic”. *2014 IEEE/RSJ international conference on intelligent robots and systems*. IEEE. 2014, 2997–3004. old.

- [30] David H Douglas és Thomas K Peucker. „Algorithms for the reduction of the number of points required to represent a digitized line or its caricature”. *Cartographica: the international journal for geographic information and geovisualization* 10.2 (1973), 112–122. old.
- [31] Shaik Abdul Nusrath Begum és KP Supreethi. „A survey on spatial indexing”. *Journal of Web Development and Web Designing* 3.1 (2018).
- [32] Antonin Guttman. „R-trees: A dynamic index structure for spatial searching”. *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*. 1984, 47–57. old.
- [33] Bentley és Ottmann. „Algorithms for Reporting and Counting Geometric Intersections”. *IEEE Transactions on Computers* C-28.9 (1979), 643–647. old. DOI: 10.1109/TC.1979.1675432.
- [34] Ibrahim Ibrahim, Wilm Decré és Jan Swevers. „Accelerated Reeds-Shepp and Under-Specified Reeds-Shepp Algorithms for Mobile Robot Path Planning”. *IEEE Transactions on Robotics* (2025).
- [35] Ioan A. Şucan, Mark Moll és Lydia E. Kavraki. „The Open Motion Planning Library”. *IEEE Robotics & Automation Magazine* 19.4 (2012). <https://ompl.kavrakilab.org>, 72–82. old. DOI: 10.1109/MRA.2012.2205651.
- [36] Abhisek Ukil, Vishal H Shah és Bernhard Deck. „Fast computation of arctangent functions for embedded applications: A comparative analysis”. *2011 IEEE International Symposium on Industrial Electronics*. IEEE. 2011, 1206–1211. old.
- [37] F. Mazzo. *Speeding up atan2f by 50x*. 2021. URL: <https://mazzo.li/posts/vectorized-atan2.html> (elérés dátuma 2025. 04. 29.).
- [38] UW Hochstrasser. *Handbook of mathematical functions, M. Abramowitz and IA Stegun, ed.* 1965.
- [39] NVidia. *asin*. 2025. URL: <https://developer.download.nvidia.com/cg/asin.html> (elérés dátuma 2025. 04. 29.).
- [40] Algorithms for Competitive Programming. *Check if point belongs to the convex polygon in $O(\log N)$* . 2023. URL: <https://cp-algorithms.com/geometry/point-in-convex-polygon.html> (elérés dátuma 2025. 04. 29.).
- [41] Frédéric Goualard. „The Ins and Outs of Solving Quadratic Equations with Floating-Point Arithmetic”. *HAL open science* (2023).

- [42] International Organization for Standardization. *ISO ISO 20900:2023*. 2023. URL: <https://www.iso.org/standard/82953.html> (elérés dátuma 2025. 04. 29.).
- [43] Ltd. China Automotive Engineering Research Institute Co. *iVista iVista 2023, China Intelligent Vehicle Index*. 2023. URL: <https://www.autoindex.org.cn/indexmarkzn/indexMark1.html> (elérés dátuma 2025. 04. 29.).
- [44] Philipp Gulde és Joachim Hermsdörfer. „Smoothness metrics in complex movement tasks”. *Frontiers in neurology* 9 (2018), 615. old.
- [45] Diederik P Kingma és Jimmy Ba. „Adam: A method for stochastic optimization”. *arXiv preprint arXiv:1412.6980* (2017).

Ábrák jegyzéke

1.1. Egy kihívást jelentő parkolási eset	7
2.1. Önvezetés komponensdiagrammja	9
2.2. Kinematikai bicikli modell	11
2.3. Dubins-útvonal és Reeds–Shepp-útvonal összehasonlítása	13
2.4. Reeds–Shepp útvonalak nem pontonként folytonosak	13
2.5. CC steer primitívekből álló útvonal	14
2.6. Hibrid A* algoritmus állapottrácsa	17
2.7. RRT* algoritmus által talált útvonalak	18
2.8. Lokális optimalizáció útvonalsimításra	19
2.9. Egy útvonal simítása Bézier-görbe segítségével	20
2.10. Egyenes és körív görbületfolytonos illesztése klotoiddal	20
2.11. Ütközésetektálás mintavételezéssel	22
3.1. Polygon-szakasz ütközésetektálás $O(\log N)$ időben	27
3.2. Egy körív által lefedett munkaterület	28
3.3. Egy körív által lefedett munkaterület köztes része	28
3.4. Távfüggő forgatás transzformáció	29
5.1. Támogatók	36
A.1. ISO 20900:2023 merőleges parkolás eset	37
A.2. ISO 20900:2023 párhuzamos parkolás eset	38
A.3. ISO 20900:2023 garázsba parkolás eset	38
A.4. iVista merőleges parkolás eset	38
A.5. iVista párhuzamos parkolás eset	39
A.6. iVista átlós parkolás eset	39
A.7. Utolsó előtti helyre parkolás esete egy parkolóházban	39
A.8. Utolsó helyre parkolás esete egy parkolóházban	40

A.9. Utolsó helyre parkolás kreatív megoldása aluldefiniált esetben	40
A.10. Utolsó helyre parkolás esete egy nagyobb járművel	40
A.11. Reeds–Shepp útvonalak osztályozása a primitívek szerint	41
A.12. Reeds–Shepp-útvonalak hossza	42

Jelölésjegyzék

$\kappa(\cdot)$	Görbület, 11
κ_{max}	Maximális görbület, 11
$\mathbf{q}_S$	Kezdőkonfiguráció, 10
$\mathbf{q}_T$	Célkonfiguráció, 10
$\mathcal{C}$	Konfigurációs tér, 9
$\mathcal{E}$	A környezet által lefedett munkaterület, 10
$\mathcal{F}$	Szabad tér, 10
$\mathcal{H}(\cdot)$	Jármű által lefedett munkaterület, 10
$\mathcal{W}$	Robot munkaterülete, 10
ψ	Kerékszög, 11
$\text{szögtáv}(\theta_1, \theta_2)$	Két irányszög távolsága, 24
θ	Jármű iránya, 11
$r(\cdot)$	Kanyarodási sugár, 11
r_{min}	Minimális kanyarodási sugár, 11